C.F.G.S. DESARROLLO DE APLICACIONES WEB

Módulo Profesional
ENTORNOS DE DESARROLLO



UD 1. DESARROLLO DE SOFTWARE

REV. 2026/27

1. ¿QUÉ ES EL DESARROLLO DE SOFTWARE? 3
1.1. Concepto de desarrollo del software. 3
1.2. La Importancia del Proceso de Desarrollo 3
2. CONCEPTOS FUNDAMENTALES: SOFTWARE Y HARDWARE 5
2.1. ¿Qué es el Software? 5
2.2. ¿Qué es el Hardware? 6
2.3. Relación Hardware-Software 7
3. EL CICLO DE VIDA DEL DESARROLLO DE SOFTWARE 10
3.1. Concepto de Ciclo de Vida del Software 10
3.2. Fases Principales del Desarrollo de una Aplicación Informática 10
3.2.1. Fase Inicial (Planificación) 11
3.2.2. Análisis (Etapa de Análisis) 11
3.2.3. Diseño 12
3.2.4. Codificación (Implementación) 13
3.2.5. Pruebas 13
3.2.6. Documentación 14
3.2.7. Explotación (Despliegue) 15
3.2.8. Mantenimiento 15
4. MODELOS DE DESARROLLO 19
4.1. Modelo de desarrollo en Cascada 19
4.2. Modelo de desarrollo Iterativo e Incremental 20
4.3. Modelos de desarrollo Ágil 21
4.3.1. SCRUM 22
5. LENGUAJES DE PROGRAMACIÓN 25
5.1. ¿Qué es un Lenguaje de Programación? 25
5.1.1. Elementos que componen un lenguaje de programación 25
5.1.2. Otros elementos importantes 25
5.2. Clasificación de Lenguajes de Programación 26
5.2.1. Según su cercanía al lenguaje humano (Nivel de Abstracción) 26
5.2.2. Según su mecanismo de traducción (Compilados, Interpretados, Mixtos) 28
5.2.3. Según su sistema de tipos (Tipado Fuerte, Tipado Débil) 30
5.2.4. Según la forma en que operan (Paradigmas de Programación) 31
5.3. Criterios para la Selección de un Lenguaje de Programación 33
6. PROCESO DE TRADUCCIÓN, MÁQUINAS VIRTUALES Y ENTORNOS DE EJECUCIÓN 35
6.1. Proceso de Traducción: Compilación e Interpretación 35
6.1.1. Diferenciación entre Traducción, Compilación e Interpretación 35

6.1.2. Fases de un Traductor (Compilador/Intérprete) 37
6.2. Códigos Fuente, Objeto y Ejecutable 38
6.3. Máquinas Virtuales y Entornos de Ejecución 38
6.3.1. Concepto de Máquina Virtual 38
6.3.2. Entornos de Ejecución (Runtime Environments) 40
6.3.3. Frameworks 41
7. HERRAMIENTAS DE APOYO AL DESARROLLO DE SOFTWARE 43
7.1. Herramientas de Desarrollo 43
7.2. Herramientas CASE (Computer Aided Software Engineering) 43
7.4. Entornos de Desarrollo Integrado (IDE) 44
7.5. Control de Versiones: Git 45
7.6. Contenedores: Docker 45
8. PERFILES DEL DESARROLLO DE SOFTWARE 47
8.1. Arquitecto de Software 47
8.2. Jefe de Proyecto 47
8.3. Analista de Sistemas 48
8.4. Analista Programador 49
8.5. Programador (o Desarrollador) 49
8.6. QA (Quality Assurance) / Testeador 50
8.8. Scrum Master 51
8.10. Diseñador UX/UI 51
8.11. Organigrama de un Equipo de Desarrollo 52

1. ¿QUÉ ES EL DESARROLLO DE SOFTWARE?

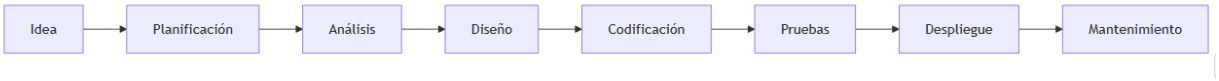
1.1. Concepto de desarrollo del software.

El **Desarrollo de Software** abarca todo el proceso que ocurre desde que se concibe una idea hasta que un programa está implementado en el ordenador y funcionando. Es una disciplina que estudia los principios y metodologías para el desarrollo y mantenimiento de sistemas software. Algunos autores consideran que el término "desarrollo de software" es más apropiado que "ingeniería de software", ya que este último implica niveles de rigor y prueba de procesos que no son siempre adecuados para todo tipo de desarrollo de software. La **Ingeniería del software** se define como la ciencia y el arte de especificar, diseñar y desarrollar programas, documentación y procedimientos operativos..

Ejemplo cotidiano

Pensemos en una aplicación de reparto de comida como Glovo o Uber Eats:

- Alguien tuvo una **idea**: "Quiero pedir comida a casa desde el móvil"
- Se analizó el **mercado** y las **necesidades** de los usuarios
- Se diseñó la **arquitectura** de la aplicación
- Se desarrolló el **código** para web y móviles
- Se probó con usuarios reales (beta testing)
- Se desplegó y hoy millones de personas la usan
- Se mantiene y mejora constantemente con nuevas funcionalidades



💡 **Dato curioso:** El término "bug" (bicho) para referirse a un error de software proviene de 1947, cuando la científica Grace Hopper encontró una polilla real atascada en un relé de la computadora Harvard Mark II.

1.2. La Importancia del Proceso de Desarrollo

El proceso de desarrollo, que al principio puede parecer una tarea simple, consta de una serie de pasos de obligado cumplimiento. Solo así se puede garantizar que los programas creados sean eficientes, fiables, seguros y respondan a las necesidades de los usuarios finales.

¿Por qué no podemos "saltarnos" pasos?

Imagina que quieres construir una casa. ¿Empezarías a colocar ladrillos sin tener los planos? ¿Sin saber dónde estarán las tuberías? ¿Sin calcular los cimientos? Obviamente no. Lo mismo ocurre con el software.

Etapa omitida	Consecuencia típica
Sin planificación	El proyecto se queda sin presupuesto a medias
Sin análisis	El software no responde a lo que necesita el cliente
Sin diseño	Código desorganizado, difícil de mantener
Sin pruebas	Errores que el usuario descubre en producción
Sin documentación	Nadie sabe cómo funciona el sistema

⚠ **Error común:** Muchos estudiantes creen que "programar" es lo más importante. ¡En realidad, programar es solo UNA fase! Un programador que no sabe analizar requisitos o diseñar estructuras de datos creará código inútil aunque funcione técnicamente.

La regla del 80/20

En desarrollo de software se cumple una regla no escrita: el **80% del tiempo** de un proyecto se dedica a **mantenimiento y evolución**, mientras que solo el **20%** corresponde al desarrollo inicial. Por eso es crucial hacer las cosas bien desde el principio.

Recursos interesantes en You Tube

Introducción

https://youtu.be/KybhDIWZj44?si=dsI6CaEr1GyxLFD6

Algoritmos y lenguajes

https://youtu.be/3NEh4kb9raU?si=mNSaPFatz52PEDhf

2. CONCEPTOS FUNDAMENTALES: SOFTWARE Y HARDWARE

2.1. ¿Qué es el Software?

El **software** es la parte intangible o lógica de un sistema informático. Es el conjunto de programas informáticos que actúan sobre el hardware para ejecutar lo que el usuario desee. Se desarrolla para llevar a cabo una tarea determinada, se comunica con el hardware y le indica qué hacer, y se encarga de traducir las instrucciones de los usuarios.

Tipos de Software

Software de Sistema

Es el software base que debe estar instalado y configurado en el ordenador para que las aplicaciones puedan ejecutarse y funcionar. Incluye el sistema operativo (como Windows, Linux, Mac OS X) y los *drivers* o controladores de dispositivos.

Ejemplos en tu día a día:

- Windows 11, macOS, Ubuntu Linux (Sistemas Operativos)
- Controladores de tu tarjeta gráfica NVIDIA/AMD
- Drivers de tu impresora HP o Canon

💡 **Ejemplo real:** ¿Alguna vez has actualizado Windows y tu impresora ha dejado de funcionar? Eso es un problema de driver: el sistema operativo cambió y el controlador necesita actualizarse para entender las nuevas órdenes. Por eso los fabricantes publican drivers periódicamente.

Software de Aplicación

Es un conjunto de programas que tienen una finalidad más o menos concreta. Ejemplos incluyen suites ofimáticas, navegadores, editores de imagen, procesadores de textos, hojas de cálculo, reproductores de música o videojuegos.

Ejemplos:

- Microsoft Word, Google Docs (procesamiento de textos)
- Excel, hojas de cálculo
- Chrome, Firefox, Edge (navegadores)
- Photoshop, GIMP (edición de imagen)
- Spotify, Netflix (entretenimiento)
- WhatsApp, Telegram (mensajería)

Software de Desarrollo (o de Programación)

Es el conjunto de herramientas que permiten desarrollar programas informáticos. Esto incluye editores, compiladores, intérpretes, entre otros.

Herramientas que usarás en DAW:

- Visual Studio Code, IntelliJ IDEA, Eclipse (editores/IDEs)
- GCC, Clang (compiladores de C/C++)

- Python interpreter
- Git (control de versiones)

Tipos según personalización

Software a medida

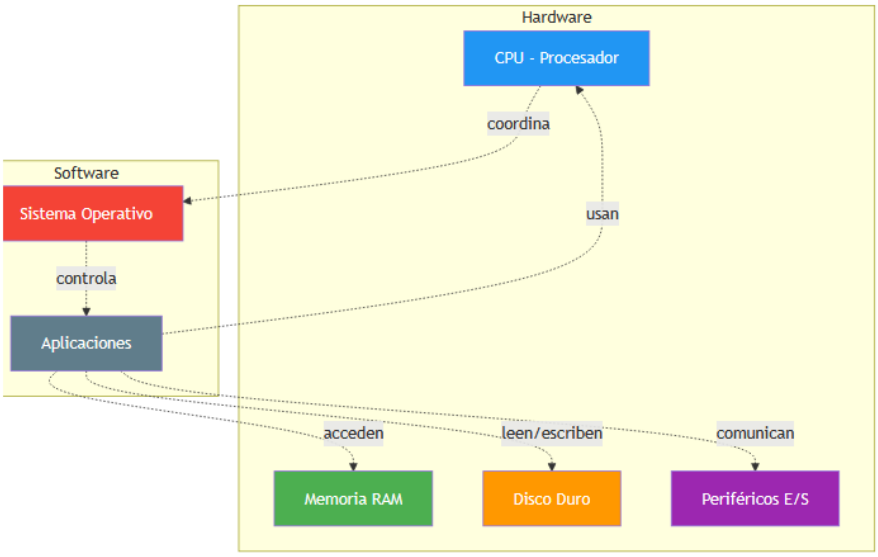
Se desarrolla según las especificaciones o requerimientos de una empresa u organismo, adaptándose a su actividad específica. Necesita tiempo de desarrollo, se adapta a necesidades concretas, puede contener errores iniciales y suele ser más costoso.

Ejemplos reales:

- Sistema de gestión de inventarios para un supermercado específico
- Aplicación de citas médicas para un hospital
- Software de contabilidad adaptado a la legislación española

Software estándar

Es un software genérico válido para cualquier cliente potencial y resuelve múltiples necesidades, a menudo incluyendo herramientas de configuración. Se compra ya desarrollado, suele tener menos errores al ser más testeado y es más barato. Sin embargo, puede incluir funciones nunca usadas o carecer de opciones importantes.



Ejemplos:

- SAP (gestión empresarial)
- Salesforce (CRM)
- Microsoft Dynamics (ERP)

📝 **Nota:** En DAW vais a crear tanto software a medida (prácticas y proyectos) como a integrar soluciones estándar (usando APIs, conectando con bases de datos existentes, etc.). Ambos enfoques son valiosos en la industria.

2.2. ¿Qué es el Hardware?

El **hardware** es el conjunto de dispositivos físicos que conforman un ordenador. Los componentes principales del sistema informático incluyen:

- **CPU (Unidad Central de Procesamiento)**: También llamada UCP (en inglés), procesador o microprocesador. Lee y ejecuta las instrucciones almacenadas en la memoria RAM, así como los datos necesarios.

- **Memoria RAM**: Almacena de forma temporal el código binario de los archivos ejecutables y los archivos de datos necesarios para la ejecución del programa. Es volátil: cuando apagas el ordenador, se borra todo lo que hay en ella.
- **Memoria ROM**: Almacena de forma permanente las instrucciones básicas para iniciar el ordenador. Contiene el **firmware** (BIOS o UEFI): un programa grabado en la placa madre que se ejecuta al encender el equipo y comprueba que todo el hardware funcione antes de cargar el sistema operativo. Es de solo lectura: no se puede modificar ni borrar durante el uso normal del equipo.
- **Disco Duro**: Almacena de forma permanente los archivos ejecutables y los archivos de datos. Se considera un periférico de Entrada/Salida (E/S).
- **Periféricos de Entrada/Salida (E/S)**: Dispositivos que permiten la comunicación entre el usuario y el ordenador.
    - **Entrada**: teclado, ratón, micrófono, cámara, escáner, pantalla táctil
    - **Salida**: pantalla, altavoces, impresora, auriculares
    - **Entrada/Salida**: pendrive, disco externo, tarjeta de red, pantalla táctil

La metáfora de la cocina

Componente	**Analogía culinaria**	**Función**
CPU	Chef	Ejecuta las tareas, procesa la información
RAM	Encimera de trabajo	Espacio temporal para trabajar
ROM	Receta de la casa	Instrucciones permanentes que el chef necesita para arrancar
Disco duro	Nevera/armario	Almacenamiento permanente
Periféricos E/S	Ventanilla de pedidos (entrada) y platos servidos (salida)	Comunicación con el usuario

2.3. Relación Hardware-Software

Existe una relación indisoluble entre hardware y software, ya que ambos necesitan estar instalados y configurados correctamente para que el equipo funcione. El software se ejecutará sobre los dispositivos físicos.

Tabla comparativa: Software vs Hardware

Característica	**Software**	**Hardware**
Naturaleza	Intangible (programas, datos)	Físico (dispositivos)
Ejemplos	Windows, Word, Chrome	CPU, RAM, disco duro
Se deteriora	No (se copia perfectamente)	Sí (se desgasta)

Característica	Software	Hardware
Se puede robar	Sí (copia ilegal)	Sí (robo físico)
Depende de	Hardware para ejecutarse	Software para funcionar
Se mide en	MB, GB, líneas de código	GHz, MB, TB

Esta relación se manifiesta desde dos puntos de vista:

Desde el punto de vista del sistema operativo

El **sistema operativo** es el encargado de coordinar el hardware durante el funcionamiento del ordenador, actuando como intermediario entre este y las aplicaciones que se están ejecutando. Todas las aplicaciones necesitan recursos hardware (tiempo de CPU, espacio en memoria RAM, tratamiento de interrupciones, gestión de dispositivos de E/S, etc.) durante su ejecución. El sistema operativo controla estos aspectos de manera "oculta" para las aplicaciones y el usuario.

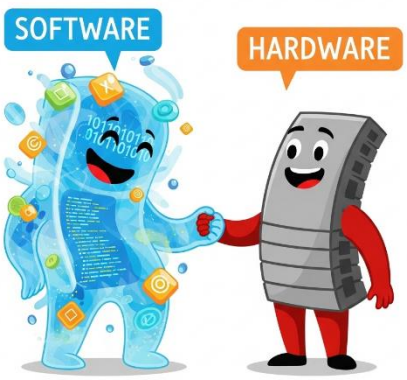
Funciones del Sistema Operativo:

- Gestión de procesos (¿qué programa se ejecuta ahora?)
- Gestión de memoria (¿cuánta RAM usa cada programa?)
- Gestión de archivos (¿dónde se guardan los datos?)
- Gestión de dispositivos (¿cómo se comunica con la impresora?)
- Seguridad (¿quién puede acceder a qué?)

Desde el punto de vista de las aplicaciones

Una aplicación es un conjunto de programas, escritos en algún lenguaje de programación. Mientras que los lenguajes de programación están diseñados para que los humanos puedan entenderlos y usarlos fácilmente, el hardware solo interpreta señales eléctricas (ausencias o presencias de tensión) que se traducen en secuencias de 0 y 1 (código binario). Esto implica que el código escrito en un lenguaje de programación debe pasar por un proceso de traducción para que el ordenador pueda entenderlo y ejecutarlo.

💡 **Dato curioso:** El código máquina de un procesador Intel i9 tiene más de 3.000 instrucciones diferentes. Cuando programas en Python o Java, estás usando abstracciones que ocultan esta complejidad.



📝 **Nota:** Esta separación entre hardware y software es lo que hace posible que puedas ejecutar el mismo programa (por ejemplo, Visual Studio Code) en Windows, Linux o macOS. El código es el mismo, pero el sistema operativo traduce tus órdenes a las señales específicas que entiende cada hardware.

💡 **Dato profesional:** En DAW, necesitarás entender esta relación para diagnosticar problemas en aplicaciones. Un error puede ser de software (bug en el código) o de hardware (falta de memoria, disco lleno).

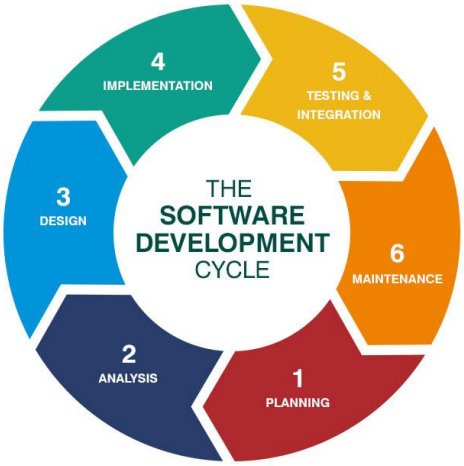
Resumen del punto:

Concepto	Descripción
Software	Parte lógica: sistemas operativos, aplicaciones, herramientas de desarrollo
Hardware	Parte física: CPU, RAM, disco, periféricos
Relación	El software necesita hardware para ejecutarse; el hardware necesita software para funcionar
SO	Intermediario que gestiona los recursos hardware para las aplicaciones

En el siguiente punto veremos el **ciclo de vida del desarrollo de software**, es decir, cómo se crea el software que se ejecuta en este hardware.

3. EL CICLO DE VIDA DEL DESARROLLO DE SOFTWARE

3.1. Concepto de Ciclo de Vida del Software



La serie de pasos a seguir para desarrollar un programa es lo que se conoce como **Ciclo de Vida del Software**. Cada etapa del ciclo de vida del software se explicará con más detalle, y el desarrollo de software es un proceso que conlleva una serie de pasos genéricos. Es un proceso que puede parecer muy complejo y que exige una gran coordinación y disciplina del grupo de trabajo que lo desarrolle.

💡 **Analogía:** El ciclo de vida del software es como el ciclo de vida de una persona: nace (se concibe la idea), crece (se desarrolla), se reproduce (se mantiene y evoluciona), y eventualmente muere (se retira). Cada etapa tiene sus propias necesidades y cuidados.

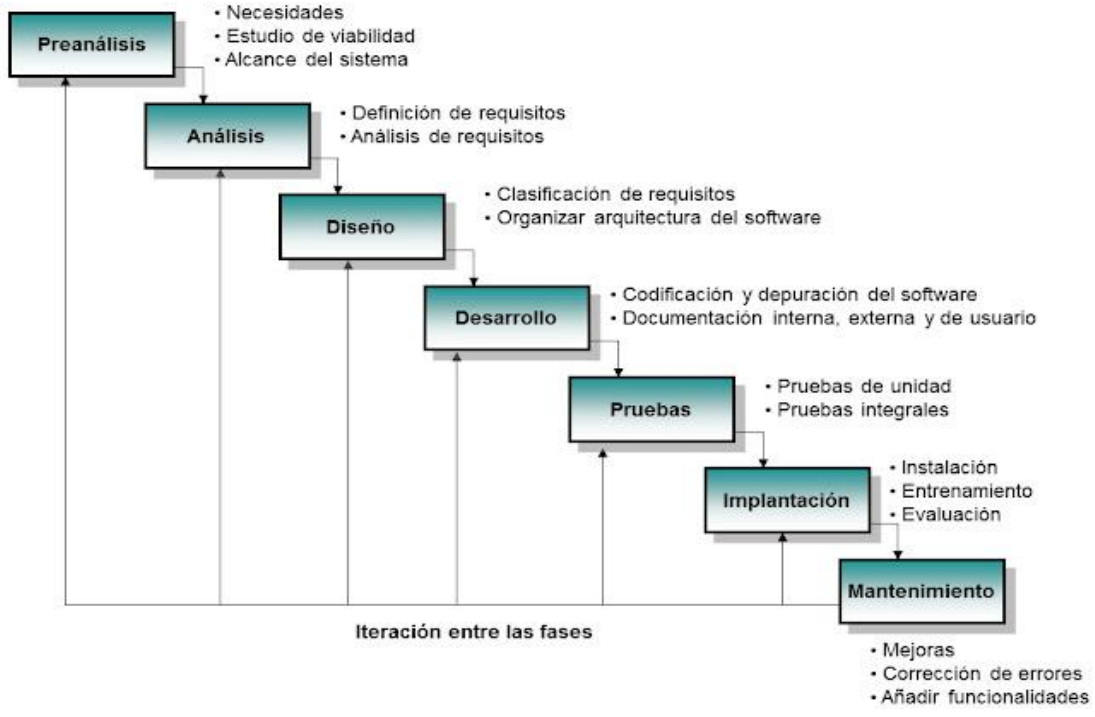
¿Por qué seguir un ciclo de vida?

Sin un ciclo de vida estructurado, los proyectos de software suelen fracasar. Según estudios de la industria:

- **El 70% de los proyectos de software fracasan** (están sobre-costados, llegan tarde o no cumplen requisitos)
- **Los errores en fases tempranas cuestan 10-100x más** si se descubren tarde
- **El mantenimiento representa el 60-80% del coste total** del software

3.2. Fases Principales del Desarrollo de una Aplicación Informática

Independientemente del modelo elegido, siempre hay una serie de etapas que se deben seguir para construir software fiable y de calidad. Las fases principales, comúnmente aceptadas, son:



3.2.1. Fase Inicial (Planificación)

En esta fase se establecen los **objetivos** del proyecto, se define su **alcance** y se realiza un **estudio de viabilidad y costes**. Es la fase más compleja, que precisa de expertos en planificación de proyectos y donde se desarrollan documentos importantes como el de viabilidad y estimación.

Preguntas clave de esta fase:

- ¿Qué queremos lograr?
- ¿Es técnicamente posible?
- ¿Tenemos recursos (tiempo, dinero, personal)?
- ¿Cuánto costará?
- ¿Cuándo estará listo?

Documentos típicos:

- Acta de constitución del proyecto
- Estudio de viabilidad
- Estimación de costes y plazos

💡 **Ejemplo real:** Antes de desarrollar Instagram, los fundadores analisaron: "¿La gente quiere compartir fotos con filtros?" "Sí, pero necesitamos servidores baratos y una app ligera". Sin este análisis, habrían invertido millones en un producto que nadie quería.

3.2.2. Análisis (Etapa de Análisis)

Esta es la primera fase y la de mayor importancia en el desarrollo del proyecto. Todo lo demás dependerá de lo bien detallada que esté, siendo también la más complicada ya que no está automatizada y depende en gran medida del analista que la realice.

En esta fase, se determina y define claramente las **necesidades del cliente** y se especifican los **requisitos** que debe cumplir el software a desarrollar. Lo fundamental es una buena comunicación entre el analista y el cliente para que la aplicación desarrollada cumpla con sus expectativas, es decir, necesitamos saber el "**qué**" hace nuestro software.

Especificación de Requisitos

La **especificación de requisitos** debe:

- Ser completa y sin omisiones.
- Ser concisa y sin trivialidades.
- Evitar ambigüedades, utilizando lenguaje formal.
- Evitar detalles de diseño o implementación.
- Ser entendible por el cliente.
- Separar requisitos funcionales y no funcionales.
- Dividir y jerarquizar el modelo.
- Fijar criterios de validación.

Tipos de Requisitos

- **Requisitos Funcionales**: Definen qué funciones tendrá que realizar la aplicación. Responden a preguntas como qué respuesta dará la aplicación ante todas las entradas o cómo se comportará en situaciones inesperadas. Por ejemplo, en una aplicación de cosmética, podría ser si desea que la lectura de productos se haga mediante códigos de barras, cómo se detallan las facturas, si se controlará el stock o si se operará con tarjetas de crédito.

Ejemplo práctico (tienda online):

ID	Requisito	Prioridad
RF-01	El sistema permitirá registrar nuevos usuarios	Alta
RF-02	El sistema mostrará catálogo de productos	Alta
RF-03	El sistema permitirá búsquedas por categoría	Media
RF-04	El sistema enviará email de confirmación tras compra	Alta

- **Requisitos No Funcionales**: Definen las características de calidad del sistema. Incluyen tiempos de respuesta del programa, legislación aplicable, tratamiento ante la simultaneidad de peticiones, etc.

Ejemplos:

- RNF-01: "El sistema responderá en menos de 2 segundos"
- RNF-02: "El sistema soportará 1000 usuarios simultáneos"
- RNF-03: "Cumple RGPD en el tratamiento de datos personales"
- **Requisitos de Información**: Detallan qué información se ofrecerá como salida y qué datos son necesarios como entrada para resolver el problema.

La culminación de esta fase es el documento de **Especificación de Requisitos del Software (ERS)**, que actúa como un contrato entre el cliente y el desarrollador.

📝 **Nota:** Los requisitos de información detallan qué datos entran y salen del sistema. Ejemplo: "El sistema recibirá: DNI, nombre, dirección. El sistema generará: factura con fecha, productos, subtotal, IVA y total.".

⚠️ **Advertencia:** No confundir requisitos funcionales (qué hace el sistema) con requisitos de diseño (cómo lo hace). Ejemplo: "El sistema guardará los datos en una base de datos PostgreSQL" es un DETALLE DE IMPLEMENTACIÓN, no un requisito funcional.

3.2.3. Diseño

Durante esta fase, una vez que ya se sabe "qué" hay que hacer, el siguiente paso es definir "**cómo**" hacerlo. Se descompone y organiza el sistema en elementos componentes que pueden ser desarrollados por separado, especificando su interrelación y funcionalidad.

Las actividades habituales incluyen el diseño arquitectónico, el diseño detallado, el diseño de datos y el diseño de la interfaz de usuario. En este punto, se deben tomar decisiones importantes como las entidades y relaciones de las bases de datos, la selección del lenguaje de programación y la elección del Sistema Gestor de Base de Datos (SGBD). Los documentos generados son más técnicos. El resultado tras el diseño arquitectónico es el **Documento de arquitectura del software**, y tras el diseño detallado, la **Especificación de módulos y funciones**.

Artefactos típicos del diseño:

- Diagramas UML (clases, secuencia, casos de uso)
- Modelo entidad-relación de la base de datos
- Maquetas de interfaces de usuario
- Especificación de APIs

💡 **Analogía:** Si la fase de análisis responde "construiré una casa de 3 habitaciones con jardín", la fase de diseño responde "la cocina estará aquí, el salón tendrá 30m², usaremos ladrillo caravista, el fontanero entrará por aquí...".

3.2.4. Codificación (Implementación)

Esta etapa consiste en transformar o traducir los resultados obtenidos a un determinado lenguaje de programación. Se escribe el código fuente de cada componente, traduciendo los algoritmos definidos en la fase de diseño. Esta tarea la realiza el programador y debe cumplir exhaustivamente con los datos impuestos en el análisis y diseño.

Características deseables del código

1. **Modularidad**: Dividido en trozos pequeños.
2. **Corrección**: Que haga lo que se le pide.
3. **Facilidad de lectura**: Para facilitar su desarrollo y mantenimiento futuro.
4. **Eficiencia**: Que haga un buen uso de los recursos.
5. **Portabilidad**: Que se pueda implementar en cualquier equipo.

Durante esta fase, el código pasa por diferentes estados (código fuente, objeto, ejecutable). El resultado de esta fase es el **Código fuente**.

📝 **Nota:** Programar no es solo "escribir código". Un buen programador:

- Lee código ajeno más que escribe código propio
- Sigue convenciones y estándares
- Comenta el "por qué", no el "qué" (el código ya dice qué hace)
- Usa control de versiones (Git)
- Pide revisiones de código (code review)

3.2.5. Pruebas

El principal objetivo de las **pruebas** es conseguir que el programa funcione incorrectamente para descubrir y corregir defectos. El programa debe ser sometido al máximo número de situaciones diferentes. Las pruebas son imprescindible para asegurar la validación y verificación del software construido.

💡 **Concepto clave — Verificación vs Validación:**

- **Verificación**: ¿Hacemos el producto bien? Comprueba que el código cumple con las especificaciones técnicas.
- **Validación**: ¿Hacemos el producto correcto? Comprueba que el software satisface las necesidades reales del usuario.

Ejemplo: Un cajero automático verifica que el código funcione (verificación), pero si el usuario no sabe usarlo porque el diseño es confuso, no está validado.

💡 **Consejo:** "Si no has encontrado un bug, es que no has probado lo suficiente."

Tipos de Pruebas

- **Pruebas Unitarias**: Prueban, una a una, las diferentes partes del software y comprueban su funcionamiento por separado.

Ejemplo: Probar que una función que suma dos números devuelve el resultado correcto.

- **Pruebas de Integración**: Se realizan una vez que las pruebas unitarias han sido exitosas, comprobando el funcionamiento del sistema completo con todas sus partes interrelacionadas.

Ejemplo: Probar que el módulo de login se conecta correctamente con la base de datos de usuarios.

- **Pruebas Funcionales**: Validan que la aplicación hace lo que tiene que hacer, a menudo con la participación del cliente.

Ejemplo: "Dado un usuario con cuenta, cuando hace login, entonces ve su panel de control".

- **Pruebas Estructurales**: Pruebas técnicas sobre el sistema (estrés, carga, integración, etc.).

Ejemplo: ¿Cuántos usuarios simultáneos aguanta el servidor antes de caer?

- **Beta Test**: La prueba final que se realiza sobre el entorno de producción, en el entorno real del cliente y bajo un funcionamiento normal de su empresa.

💡 **Ejemplo real:** Netflix prueba nuevas funciones con un 1% de usuarios antes de lanzarlas globalmente. Si detectan problemas, corrigen y prueban con otro 1%. Esto minimiza el riesgo de un fallo masivo.

Los resultados de las pruebas de unidades son **Módulos utilizables**, y de las pruebas de integración, un **Sistema utilizable**. Las pruebas del sistema culminan con un **Sistema aceptado**.

📝 **Nota:** En DAW trabajaréis con frameworks de testing como JUnit (Java). NO es opcional, es parte del trabajo profesional.

3.2.6. Documentación

La **documentación** es vital para el desarrollo y mantenimiento del software. Todas las etapas en el desarrollo de software deben quedar perfectamente documentadas. Una correcta documentación permitirá la reutilización de parte de los programas en otras aplicaciones, especialmente si se desarrollan con diseño modular.

Tipos de Documentos

- **Guía Técnica (o Manual Técnico)**: Dirigida al personal técnico (analistas y programadores). Refleja el diseño, la codificación de los programas y las pruebas realizadas. Su objetivo es facilitar el desarrollo, las correcciones y el mantenimiento futuro.

Contenido típico: Diagramas de arquitectura, Diagramas y diccionarios de datos, manual de despliegue.

- **Guía de Uso (o Manual de Usuario)**: Dirigida a los usuarios finales (clientes). Describe la funcionalidad de la aplicación, cómo empezar a ejecutarla y ejemplos de uso. Su objetivo es dar a los usuarios toda la información necesaria para utilizar la aplicación.

Contenido: Capturas de pantalla, tutoriales paso a paso, preguntas frecuentes.

- **Guía de Instalación (o Manual de Instalación)**: Dirigida al personal informático responsable de la instalación. Detalla los requerimientos software de la aplicación y la solución a posibles problemas. Su objetivo es garantizar una implantación segura, confiable y precisa.

Contenido: Requisitos de sistema, pasos de instalación, configuración, resolución de problemas.

El resultado final es la **Documentación técnica y de usuario**.

⚠ **Advertencia:** "Sin documentación, el código es tan útil como un contrato en un idioma que no entiendes."

3.2.7. Explotación (Despliegue)

La **explotación** es la fase en que los usuarios finales conocen la aplicación y comienzan a utilizarla. Implica la instalación, puesta a punto y funcionamiento de la aplicación en el equipo final del cliente. En esta fase, los programas son transferidos al computador del usuario, configurados y verificados. Es recomendable que los clientes estén presentes durante la instalación. También se pueden llevar a cabo las Beta Test en los equipos del cliente bajo cargas normales de trabajo. La configuración puede ser realizada por los propios usuarios con la guía de instalación o programarse automáticamente si el software es sencillo. Es un momento crítico del proyecto tenerlo todo preparado antes de la presentación al cliente.

Términos relacionados:

- **Deploy**: Despliegue de la aplicación en producción
- **Release**: Nueva versión puesta a disposición de usuarios
- **Entorno de producción**: Donde los usuarios reales usan el sistema

💡 **Dato:** Hoy en día, con metodologías DevOps, el despliegue puede ser automático (CI/CD). Aplicaciones como Netflix despliegan cientos de veces al día sin intervención humana.

3.2.8. Mantenimiento

La etapa de **mantenimiento** es la más larga de todo el ciclo de vida del software. Por su naturaleza, el software es cambiante y deberá actualizarse y evolucionar con el tiempo, adaptándose a mejoras de hardware y nuevas situaciones. Siempre surgen errores y la necesidad de nuevas versiones. El mantenimiento se define como el proceso de control, mejora y optimización del software.

Tipos de Mantenimiento

- **Correctivo**: Para corregir defectos o fallos encontrados en el software.

Ejemplo: "El botón de login no funciona en Firefox versión 120".

- **Perfectivo**: Para mejorar la funcionalidad existente del software.

Ejemplo: "El formulario de registro ahora muestra mensajes de error en tiempo real".

- **Evolutivo**: Para añadir nuevas funcionalidades solicitadas por el cliente o expansiones de código.

Ejemplo: "Añadir integración con PayPal para pagos".

- **Adaptativo**: Para ajustar el software a nuevos entornos, tendencias del mercado o componentes hardware.

Ejemplo: "Hacer la app compatible con iOS 17".

Tipo	Pregunta clave	Ejemplo	Cuándo ocurre
Correctivo	¿Qué está roto?	Botón login no funciona en Firefox	Cuando hay un bug
Perfectivo	¿Qué puede mejorar?	Formulario con errores en tiempo real	Cuando el usuario pide mejoras
Evolutivo	¿Qué falta?	Añadir pago con PayPal	Cuando hay nuevos requisitos
Adaptativo	¿Dónde necesita funcionar?	Compatible con iOS 17	Cuando cambia el entorno

Los resultados del mantenimiento son **Informes de errores y control de cambios**.

📝 **Nota:** Cuando heredéis código de otros desarrolladores (en empresas o en GitHub), el 80% de vuestro trabajo será mantenimiento. Por eso es crucial que el código esté bien documentado y sea mantenible.

3.2.9. Retirada del Software

Esta fase ocurre cuando el software ha llegado al **final de su vida útil** y ya no resulta rentable seguir ampliándolo o manteniéndolo. En este punto, el ciclo puede comenzar de nuevo, ya sea comprando un nuevo software o desarrollando uno a medida.

Señales de que un software debe retirarse:

- Los costes de mantenimiento superan los beneficios
- El hardware ya no es compatible
- Las tecnologías usadas están obsoletas
- El software no cumple nuevas regulaciones legales

Ejemplos reales:

- Windows XP: Soporte terminado en 2014, pero muchas empresas lo usaron hasta 2019
- Flash Player: Adobe lo discontinuó en 2020 por razones de seguridad
- MySpace: En su momento fue la red social más grande; ahora está casi abandonada

Mini-Proyecto: Calculadora de Calificaciones

Para ver cómo el ciclo de vida se aplica en la práctica, vamos a seguir las 9 fases con un proyecto sencillo: una app de consola en C# que calcule la media de un alumno.

Fase 1 — Planificación

- **Objetivo:** Crear una app que calcule la media de 3 notas y muestre si el alumno aprueba.
- **Viabilidad:** Se puede hacer en C# de consola. Un solo programador, 1 día.
- **Coste estimado:** 0 € (proyecto de aprendizaje).

Fase 2 — Análisis

- **Requisitos funcionales:** El usuario introduce 3 notas. El sistema calcula la media. El sistema muestra el resultado y si aprueba (≥5) o suspende (<5).
- **Requisitos no funcionales:** Respuesta instantánea, mensaje claro en español.
- **ERS simplificado:**

- Entrada: 3 números decimales (notas)
- Salida: Media numérica + texto "Aprobado" o "Suspenso"

Fase 3 — Diseño

```
// Diseño de la estructura
double nota1 = PedirNota("Primera nota");
double nota2 = PedirNota("Segunda nota");
double nota3 = PedirNota("Tercera nota");
double media = CalcularMedia(nota1, nota2, nota3);
MostrarResultado(media);
```

Fase 4 — Codificación

```
Console.Write("Primera nota: ");
double nota1 = Convert.ToDouble(Console.ReadLine());
Console.Write("Segunda nota: ");
double nota2 = Convert.ToDouble(Console.ReadLine());
Console.Write("Tercera nota: ");
double nota3 = Convert.ToDouble(Console.ReadLine());

double media = (nota1 + nota2 + nota3) / 3;
Console.WriteLine($"Media: {media:F1}");

if (media >= 5)
    Console.WriteLine("Aprobado");
else
    Console.WriteLine("Suspenso");
```

Fase 5 — Pruebas

Prueba	Entrada	Salida esperada	Resultado
Notas aprobadas	7, 8, 6	Media: 7.0, Aprobado	✅
Notas suspensas	3, 4, 2	Media: 3.0, Suspenso	✅
Nota límite	5, 5, 5	Media: 5.0, Aprobado	✅
Decimales	6.5, 7.3, 8.1	Media: 7.3, Aprobado	✅

Fase 6 — Documentación

```
# Calculadora de Calificaciones
## Uso
1. Ejecutar el programa
2. Introducir 3 notas (decimales separados por coma)
3. Ver la media y si se aprueba
```

Fase 7 — Explotación

```
dotnet run
```

El programa se ejecuta en la consola. Funciona en cualquier sistema operativo con .NET instalado.

Fase 8 — Mantenimiento

- **Evolutivo:** Añadir más de 3 notas (introducir notas hasta que el usuario escriba "fin").
- **Perfectivo:** Mostrar también la nota más alta y la más baja.
- **Correctivo:** Si el usuario introduce una letra en vez de un número, el programa falla. Hay que añadir validación.

Fase 9 — Retirada

Cuando el alumno ya no la necesite (o la mejore con una app web), se retira esta versión de consola.

📝 **Nota:** Este proyecto parece simple, pero tiene todas las fases. En proyectos reales, cada fase es más compleja, pero el proceso es el mismo. Lo que hacéis aquí con 20 líneas de código, en una empresa se hace con 20.000 y un equipo de 10 personas.

Resumen del punto:

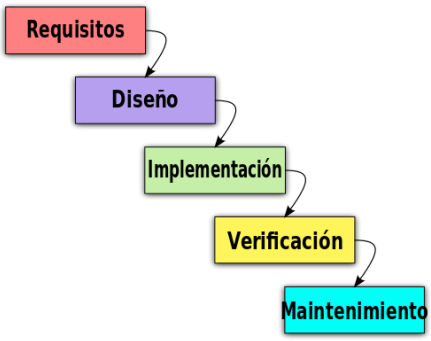
Fase	Objetivo	Resultado
Planificación	Definir objetivos y viabilidad	Estudio de viabilidad
Análisis	Conocer las necesidades del cliente	ERS (Especificación de Requisitos)
Diseño	Definir cómo se hará	Arquitectura y especificación
Codificación	Escribir el código	Código fuente
Pruebas	Encontrar errores	Sistema aceptado
Documentación	Explicar cómo funciona	Manuales
Explotación	Poner en producción	Aplicación funcionando
Mantenimiento	Evolucionar y corregir	Nuevas versiones
Retirada	Final de vida útil	Fin del ciclo

En el siguiente punto veremos los **modelos y metodologías de desarrollo**, es decir, cómo se organizan estas fases en diferentes enfoques de trabajo.

4. MODELOS DE DESARROLLO

4.1. Modelo de desarrollo en Cascada

Es el más básico de todos los modelos y ha servido como bloque de construcción para los demás paradigmas de ciclo de vida. Está basado en el ciclo convencional de una ingeniería (Se comienza por el principio) y su visión es muy simple: **el desarrollo de software se debe realizar siguiendo una secuencia de fases**.



Cada etapa tiene un conjunto de metas bien definidas y las actividades dentro de cada una contribuyen a la satisfacción de metas de esa fase. Es un modelo de ciclo de vida usado en el software de hace varias décadas, aunque sigue siendo un modelo a seguir ante cierto tipo de desarrollos.

El modelo en cascada abarca las siguientes etapas:

1. ***Análisis***: Ya que el software es siempre parte de un sistema mayor, la fase de análisis comienza con el **Análisis de los requisitos Sistema**, donde se establecen los requisitos de todos los elementos del sistema. Uno de estos elementos es los requisitos de software, que nos lleva al **Análisis de los requisitos de software**. Es el proceso de recopilación de los requisitos que se centra e intensifica especialmente en el software.
2. ***Diseño***: En esta etapa se traducen los requisitos en una representación de software. El diseño se puede basar en dos tipos: **diseño estructurado** y **diseño orientado a objetos**.
   - El diseño estructurado se compone de 4 partes: La **estructura de los datos**, donde se definen las estructuras de datos que se usarán para implementar el software. **La arquitectura del software**, que se centra en la representación de la estructura del software, sus propiedades e interacciones. El **diseño procedimental** se encarga de traducir los elementos estructurales de la arquitectura del software con suficiente nivel de detalle como para que sirva de guía en la generación de código. Por último, **el diseño de la interfaz** describe cómo se comunica el software consigo mismo, con las partes de las que está compuesto, y con los usuarios.
   - El diseño orientado a objetos se centra en el diseño del subsistema, el diseño de las clases y objetos y su jerarquía, la colaboración entre los objetos y las responsabilidades que hay entre clases. Es muy común realizar este diseño empleando el lenguaje de modelado **UML** (*Unified Modeling Language*), el cual está basado en diagramas.

3. ***Codificación o implementación***: el diseño debe traducirse en una forma legible para la máquina. Si el diseño se realiza de una manera detallada, la codificación puede realizarse mecánicamente.
4. ***Pruebas o verificación***: una vez que se ha generado el código comienza la prueba del programa. La prueba se centra en la lógica interna del software y en las funciones externas, realizando pruebas que aseguren que la entrada definida produce los resultados que realmente se requieren.

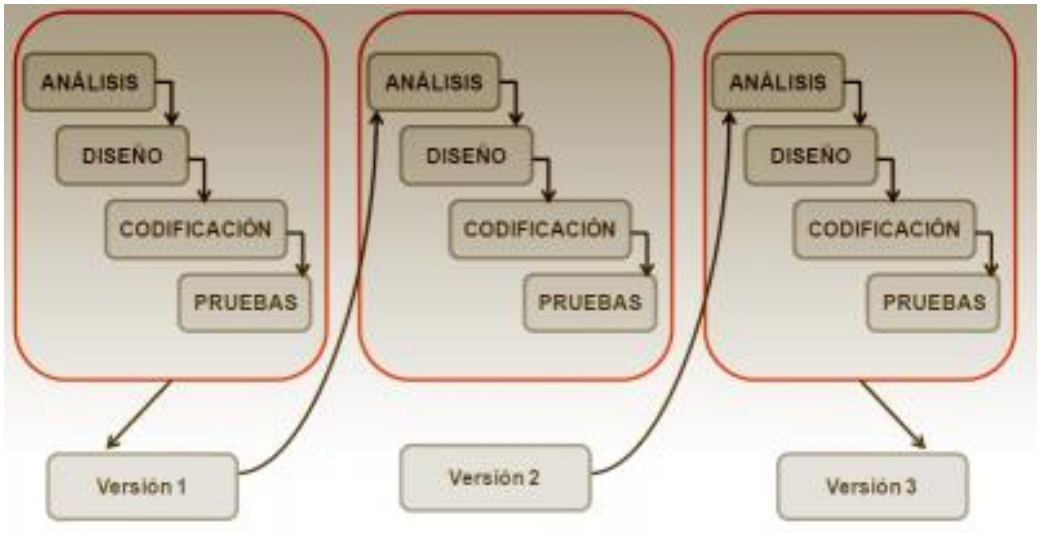
5. ***Mantenimiento*:** el software sufrirá cambios después de que se entrega al cliente. Los cambios ocurrirán debidos a que se haya encontrado errores, a que el software deba adaptarse a cambios del entorno externo (sistema operativo o dispositivos periféricos) o a que el cliente requiera ampliaciones funcionales o del rendimiento.

En el modelo vemos una ventaja evidente y radica en su sencillez, ya que sigue los pasos intuitivos necesarios a la hora de desarrollar el software. Pero el modelo se aplica en un contexto, así que debemos atender también a él y saber que:

- Los proyectos reales raramente siguen el flujo secuencial que propone el modelo. Siempre hay iteraciones y se crean problemas en la aplicación del paradigma.
- Es difícil para el cliente establecer todos los requisitos antes de comenzar.
- El cliente no puede ver una versión operativa del proyecto hasta las etapas finales. **Un error no detectado hasta que el programa esté funcionando puede ser desastroso**.

4.2. Modelo de desarrollo Iterativo e Incremental

Se basan en realizar una iteración de varios ciclos de vida en cascada realimentados. En cada iteración se entrega una pequeña parte del software funcional llamada “incrementos”. En general cada incremento se crea sobre aquel que ya ha sido entregado.



En la imagen se observa de forma iterativa distintas fases de desarrollo en cascada para la obtención de un nuevo incremento (versión del programa).

Como ejemplo de software desarrollado bajo este modelo se puede considerar un procesador de textos. En el primer incremento se desarrollan funciones básicas de gestión de archivos y de producción de documentos; en el segundo incremento se desarrollan funciones gramaticales y de corrección ortográfica, en el tercer incremento se desarrollan funciones de paginación, etc.

Ventajas:

- No se necesitan conocer todos los requisitos al comienzo
- Permite la entrega temprana de partes operativas de software.
- Las entregas facilitan la realimentación de las siguientes.

Se recomienda cuando los requisitos o el diseño no están completamente definidos y es posible que haya grandes cambios.

Los procesos ágiles se basan en este modelo, con iteraciones cortas.

4.3. Modelos de desarrollo Ágil

La visión ágil del desarrollo es una respuesta a los fracasos y las frustraciones del modelo en cascada. Contempla un enfoque para la toma de decisiones y la forma de organización en los proyectos de software, basándose en los modelos de desarrollo iterativo e incremental, con iteraciones cortas (semanas) y sin que dentro de cada iteración tenga que haber fases lineales.

En el 2001, 17 representantes de las nuevas tecnologías y el desarrollo de software se juntaron para discutir sobre el desarrollo de software. De esa reunión surgió el Manifiesto Ágil:

"Estamos descubriendo mejores maneras de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de esta experiencia hemos aprendido a valorar:

- **Individuos e interacciones** sobre procesos y herramientas
- **Software funcionando** sobre documentación extensiva
- **Colaboración con el cliente** sobre negociación contractual
- **Respuesta ante el cambio** sobre seguir un plan

Esto es, aunque los elementos a la derecha tienen valor, nosotros valoramos por encima de ellos los que están a la izquierda."

Estas ideas se explican en los 12 principios para el desarrollo ágil de software.

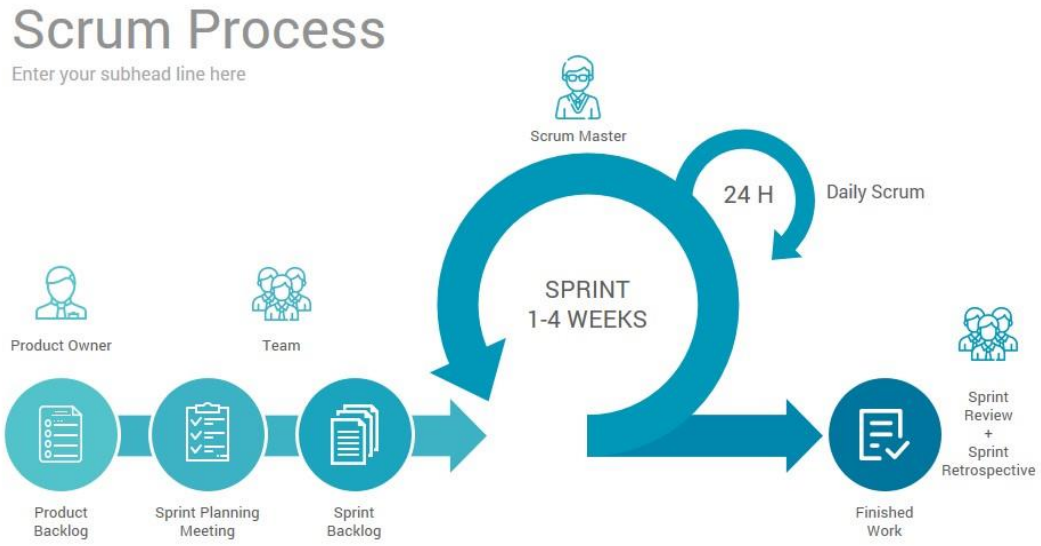
¿En qué consiste el desarrollo ágil? Los 12 principios.

1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor.
2. Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos Ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.
3. Entregamos software funcional frecuentemente, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto posible.
4. Los responsables de negocio y los desarrolladores trabajamos juntos de forma cotidiana durante todo el proyecto.
5. Los proyectos se desarrollan en torno a individuos motivados. Hay que darles el entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.
6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.
7. El software funcionando es la medida principal de progreso.
8. Los procesos Ágiles promueven el desarrollo sostenible. Los promotores, desarrolladores y usuarios debemos ser capaces de mantener un ritmo constante de forma indefinida.
9. La atención continua a la excelencia técnica y al buen diseño mejora la Agilidad.
10. La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.
11. Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados.
12. A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo para a continuación ajustar y perfeccionar su comportamiento en consecuencia.

El abanico de metodologías ágiles es amplio, existiendo métodos para organizar equipos o proyectos, y técnicas para escribir y mantener el software. Tanto las técnicas de desarrollo de software como las de organización de proyectos se pueden combinar siendo de las más famosas la programación extrema junto con la gestión Scrum.

4.3.1. SCRUM

Es una técnica de *gestión de proyectos* englobada dentro del paradigma de desarrollo ágil. No se aplica únicamente al desarrollo de software, sino a la gestión de cualquier tipo de proyectos. Como las demás, se basa en el desarrollo **iterativo e incremental** en el que el proyecto se planifica en diversos bloques temporales (un mes, normalmente) llamados **Sprints**. Cumple con las características del desarrollo ágil. La metodología Scrum se organiza de la siguiente forma:



FASES

- Reunión inicial

El cliente nos indica qué es lo que quiere. De esta reunión entre el cliente y el *Scrum Master* surge la lista de requisito del producto (*Product Backlog*). Se realiza una sola vez al inicio del proyecto, a diferencia de las demás fases que se realizan en cada Sprint.

Planificación de la iteración (**Sprint Planning**)

Es una reunión en la se habla con el cliente quien indica los requisitos prioritarios de la iteración (Sprint), se plantean las dudas y se traducen dichos requisitos a tareas (Sprint Backlog) que deben ser realizadas por cada uno de los miembros del equipo de desarrollo durante el Sprint actual. Estas tareas se incorporan a un panel que controlará su estado de realización (por hacer, en progreso, terminado, pendiente, etc).

Ejecución de la iteración (**Sprint**)

Es el bloque de tiempo, iteración, en el cuál se realizan la tareas (2-4 semanas), siendo recomendable que siempre tengan la misma duración y la duración esté definida por el grupo en base a su experiencia.

Reunión diaria del equipo (**Daily Scrum Meeting**)

Cada día se celebra una reunión rápida (no más de 10 minutos, siempre puntual y suele ser de pié) en la que se habla del estado de las tareas del Sprint (Sprint Backlog), y en el panel en el que las tenemos indicadas las marcamos como

terminadas, bloqueadas, en progreso, etc. Solo se habla del estado de las cosas y solo los involucrados pueden hablar.

Revisión de la entrega (**Sprint Review**)

Es una reunión que se produce al final del Sprint en la que se presenta al cliente el incremento (parte del producto pactada) realizado durante el Sprint. En ella el cliente puede ver cómo han sido desarrollados los requisitos que indicó para el Sprint, si se cumplen las expectativas, y entender mejor qué es lo que necesita.

- Retrospectiva (**Sprint Retrospective**)

Cuando se ha terminado un *Sprint*, se comprobarán, medirán y discutirán los resultados obtenidos en el periodo anterior a través de las impresiones de todos los miembros del equipo. El propósito de la retrospectiva es realizar una mejora continua del proceso.

ROLES

- Cliente (**Producto Owner**)

Es quién plantea los objetivos y requisitos y ayuda al usuario a escribir las **historias de usuario** que se incorporarán al *Sprint Backlog*, definiendo las prioridades. Representa a todas las personas interesadas en el proyecto (usuarios finales, promotores, etc). Es quién indica qué quiere en total y qué quiere en cada Sprint.

- Facilitador (**Scrum Master**)

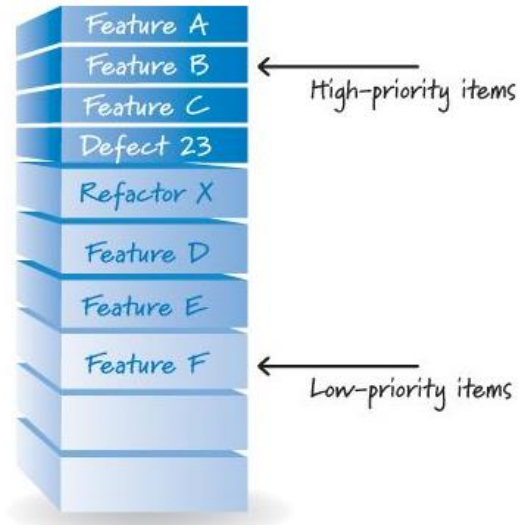
Se asegura de que se sigan los valores y principios ágiles, de la existencia de las listas de requisitos para cada iteración, facilita las reuniones, quitar los impedimentos que se van encontrando en el camino y proteger y aislar al equipo de interrupciones.

- Equipo (**Team**)

Tienen un objetivo común, se auto-organizan, comparten la responsabilidad del trabajo que realizan, y cuyos miembros confían entre ellos. 5-9 personas es lo ideal, aunque si hay más se deben hacer más equipos

HERRAMIENTAS

- Lista de requisitos priorizada (**Product Backlog**)



Es un documento abierto que representa la lista de objetivos o requisitos, y expectativas del cliente respecto a los objetivos y entregas del producto. Solo puede ser modificado por el product owner (cliente) aunque con ayuda del equipo y el "scrum master", quienes proporcionan el coste estimado tanto del valor de negocio como del esfuerzo de desarrollo. Representa el qué va a ser construido en su totalidad.

- Lista de tareas pendientes del Sprint (**Sprint Backlog**)

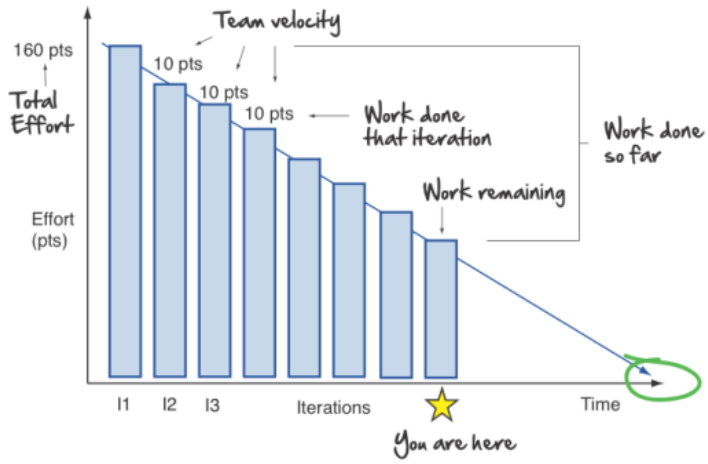
Lista de tareas que el equipo elabora en la reunión al inicio del Sprint,Sprint Planning, como plan para completar los requisitos para la iteración. Los requisitos tienen que estár claramente detallados en tareas (historias de usuario), a diferencia del Product Backlog. No se deben agregar requisitos al Sprint Backlog a menos que su falta

amenace al éxito del proyecto. Representa el cómo el equipo va a implementar los requisitos.

ID	Status	Task	Task Type	Priority Lvl.	Est. Time (hours)	Actual Time (hours)	Owner of Task
1	Complete	Finish project proposal and decide on a topic	Planning	Very High	1	1.5	Jennifer
2	Complete	Set up MySQL and PhpMyAdmin	Content	Very High	0.5	1	Jennifer
3	Complete	Create online portfolio using Wordpress	Content	Very High	0.5	0.5	Jennifer
4	Complete	Work on website wireframe and layout	Design	Very High	2	0.5	Jennifer
5	Complete	Code HTML/CSS for the pages	Feature	Very High	4	3	Jennifer
6	Complete	Decide what pages must be created for the website	Planning	High	1	0.5	Jennifer
7	Complete	Create and begin formating the other pages	Feature	High	8	3	Jennifer
8	Complete	Code some sort of navigation so users can navigate through the different pages	Feature	High	2	1	Jennifer
9	In Progress	Do research on my topic of clean drinking water	Content	High	10	TBA	Jennifer
10	In Progress	Add information that I've gathered in my research to my website	Content	Medium	2	TBA	Jennifer
11	In Progress	Integrate a 3rd party service such as Google Maps	Feature	Medium	1.5	TBA	Jennifer
12	In Progress	Find / take own pictures to use	Design	Low	1	TBA	Jennifer
13	In Progress	Find / create video(s) to incorporate	Feature	Low	1	TBA	Jennifer
14	In Progress	Create graphics and visuals using Photoshop	Design	Low	1.5	TBA	Jennifer
15	In Progress	Look into incorporating audio to the website	Feature	Low	1	TBA	Jennifer

- Gráficos de trabajo pendiente (**Burndown Chart**)

Es un gráfico de trabajo pendiente a lo largo del tiempo, en el que se muestra la velocidad a la que se están completando los requisitos. También ayuda a intuir si el equipo terminará en el tiempo estimado. Lo normal es que la línea que une todos los sprints completados sea descendente, pero s se añaden nuevos requisitos durante el proceso, puede ser ascendente.



Scrum permite conocer en un vistazo rápido cual es el estado general de las cosas, detectando rápidamente qué tareas se han quedado atascadas o qué equipos no están rindiendo al nivel que se esperaba. Es un método de desarrollo ágil ideal para entornos con mucha incertidumbre en cuanto al trabajo a realizar, en los que las tareas cambian muy rápidamente y son susceptibles de olvidarse.

5. LENGUAJES DE PROGRAMACIÓN

5.1. ¿Qué es un Lenguaje de Programación?

Un **lenguaje de programación** es un idioma creado de forma artificial, formado por un conjunto de símbolos y normas que se aplican sobre un alfabeto para obtener un código que el hardware de la computadora pueda entender y ejecutar. Son los instrumentos que tenemos para que el ordenador realice las tareas que necesitamos. Es un lenguaje formal que proporciona un conjunto de instrucciones que permiten a un programador escribir secuencias de comandos, que son interpretadas por una máquina, para producir un comportamiento deseado.

💡 **Analogía:** Un lenguaje de programación es como un puente entre tu mente (donde tienes ideas) y el ordenador (que solo entiende 0s y 1s). Sin ese puente, no hay comunicación posible.

5.1.1. Elementos que componen un lenguaje de programación

- **Alfabeto (Léxico)**: Es el conjunto finito de símbolos permitidos y palabras especiales, el vocabulario del lenguaje.
- **Sintaxis**: Son las normas de construcción permitidas de los símbolos y palabras del lenguaje. Se refiere a las reglas que rigen la estructura de las declaraciones y expresiones válidas.
- **Semántica**: Es el significado de las construcciones. Define las acciones que se llevarán a cabo con las combinaciones de los símbolos.

Ejemplo de los tres componentes:

```
// Léxico: palabras como 'void', 'Console', 'saludar'

// Sintaxis: estructura 'void Saludar(string nombre)' seguida de llaves

// Semántica: lo que ocurre cuando se ejecuta la función


void Saludar(string nombre)

{

   Console.WriteLine($"Hola, {nombre}!");

}
```

📝 **Nota:** La sintaxis es como la gramática de un idioma. Si dices "Yo hambre tengo" en español, se entiende pero no es correcto. Lo mismo pasa en programación: if (x > 5 sin cerrar el paréntesis causa error de sintaxis.

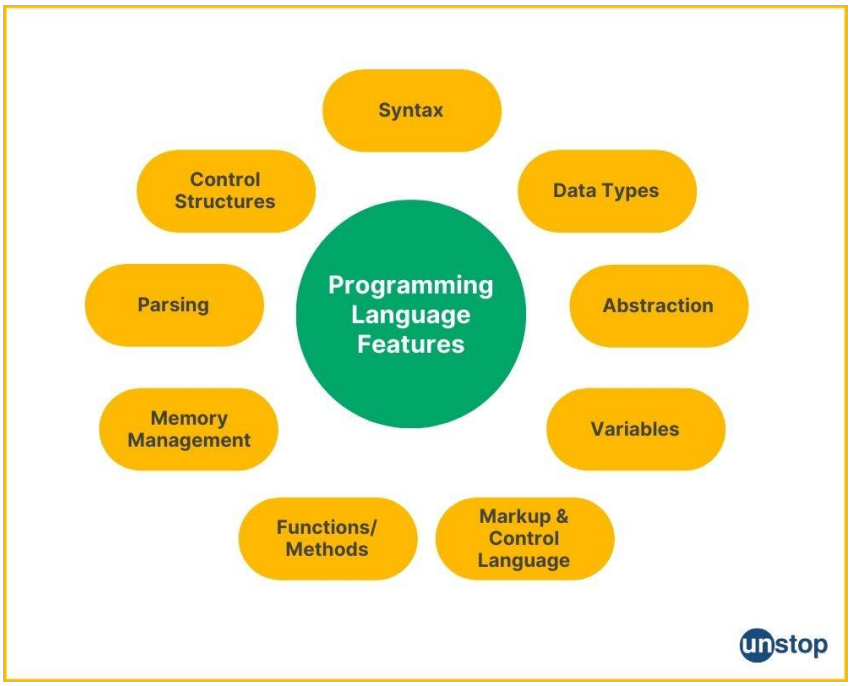
5.1.2. Otros elementos importantes

- **Tipos de Datos**: Los diferentes tipos de valores que pueden ser representados y manipulados (enteros, flotantes, caracteres, cadenas, booleanos). Definen el conjunto de valores válidos y las operaciones permitidas para ellos, así como el **espacio de memoria** que ocupan.
- **Variables**: Símbolos que representan valores en el programa y tienen tipos asociados.
- **Operadores**: Símbolos que representan operaciones específicas (por ejemplo, + para adición, == para comparación).

- **Control de Flujo**: Estructuras que determinan el orden de ejecución de las instrucciones (condicionales como if, switch; bucles como for, while).
- **Subrutinas y Funciones**: Bloques de código que pueden ser definidos y llamados por nombre, permitiendo la reutilización y modularidad.
- **Comentarios**: Notas que los programadores dejan en el código para explicar su funcionamiento, no son ejecutados.

Analogía del código:

Elemento	Analogía
Léxico	Palabras del diccionario
Sintaxis	Reglas gramaticales
Semántica	Significado de las oraciones
Variables	Cajas donde guardamos cosas
Tipos de datos	Tipo de cosa en la caja
Operadores	Acciones (+, -, *, /)
Control de flujo	Direcciones de tráfico
Funciones	Recetas de cocina reutilizables



5.2. Clasificación de Lenguajes de Programación

Los lenguajes se pueden clasificar de muchas formas. Las más importantes para DAW son:

5.2.1. Según su cercanía al lenguaje humano (Nivel de Abstracción)

Los lenguajes han evolucionado siempre hacia una mayor usabilidad y cercanía al razonamiento humano.

Lenguajes de Bajo Nivel

Son lenguajes totalmente dependientes de la máquina, lo que significa que un programa desarrollado en ellos no puede ser migrado o utilizado en otras máquinas. Aprovechan al máximo las características del hardware.

- **Lenguaje Máquina**: Sus instrucciones son combinaciones de unos y ceros (código binario). Es el único lenguaje que el ordenador entiende directamente (no necesita traducción). Fue el primer lenguaje utilizado y es único para cada procesador (no portable). Es rápido, pero difícil de manejar, con códigos fuente enormes y detección de fallos casi imposible. Hoy día nadie programa en este lenguaje.

Ejemplo de código máquina (x86):

```
10110000 01100111  (mov al, 01100111b)
10110011 00000001  (mov bl, 00000001b)
00000010 11011000  (add al, bl)
```

- **Lenguaje Ensamblador**: Sustituyó al lenguaje máquina, utilizando mnemotécnicos (instrucciones complejas) en lugar de unos y ceros. Necesita traducción al lenguaje máquina para ejecutarse. Sus instrucciones hacen referencia a la ubicación física de los archivos en el equipo. Es difícil de utilizar. Como ventaja frente al código máquina, los códigos fuente eran más cortos y ocupaban menos memoria.

Ejemplo de ensamblador x86:

```
MOV AX, 5       ; Mover el valor 5 al registro AX
MOV BX, 3       ; Mover el valor 3 al registro BX
ADD AX, BX      ; Sumar AX + BX
```

Lenguajes de Medio Nivel

Este término no es universalmente aceptado, pero se refiere a lenguajes que se encuentran en un punto medio. Pueden acceder a los registros del sistema y trabajar con direcciones de memoria (características de bajo nivel), y a la vez realizar operaciones de alto nivel. Un ejemplo es C.

C permite acceso directo a memoria con punteros (como los lenguajes de bajo nivel) pero también tiene estructuras de control, funciones y bibliotecas estándar (como los de alto nivel). Por eso se clasifica como medio nivel.

Ejemplo en C:

```c
#include <stdio.h>

int main() {
    int numero = 42;
    printf("El número es: %d\n", numero);
    return 0;
}
```

}

Lenguajes de Alto Nivel

Se encuentran más cercanos al lenguaje natural que al lenguaje máquina, y son independientes de la arquitectura del ordenador. Permiten al programador olvidarse del funcionamiento interno de la máquina. Utilizan sentencias y órdenes derivadas del idioma inglés. Necesitan un traductor para ser entendidos por la máquina. Incorporan librerías y funciones predeterminadas, y suelen ofrecer *frameworks*. La mayoría de los lenguajes de programación actuales se engloban en esta categoría. Ejemplos incluyen C#, C++, Java, Python, JavaScript, PHP.

Ejemplo en C#:

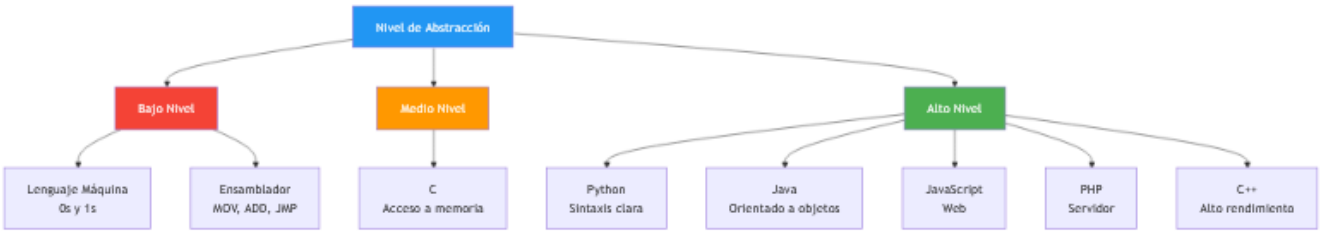
```
int numero = 42;
Console.WriteLine($"El número es: {numero}");
```



Comparativa con otros lenguajes:

```
# Python
numero = 42
print(f"El número es: {numero}")
```

```
// Java
int numero = 42;
System.out.println("El número es: " + numero);
```



💡 **Dato:** El primer lenguaje de alto nivel fue Fortran (1957), creado por John Backus en IBM. Permitía escribir fórmulas matemáticas de forma casi natural. Fortran todavía se usa hoy en cálculos científicos de alto rendimiento.

5.2.2. Según su mecanismo de traducción (Compilados, Interpretados, Mixtos)

La obtención de código binario ejecutable se realiza mediante compilación o interpretación.

Lenguajes Compilados

Necesitan un **compilador** que traduce el código fuente a código binario (código objeto) en un solo paso. Se ejecutan muy eficientemente. La principal desventaja es que es necesario compilar cada vez que el código fuente es modificado. Precisan de un programa enlazador (*linker*) que une el código objeto con el código objeto de librerías. El código es más seguro, ya que el código fuente no es directamente accesible. Ejemplos incluyen C y C++.

Proceso:

Código Fuente (.c) → Compilador → Código Objeto (.o) → Enlazador → Ejecutable (.exe)

Ventajas:

- Ejecución muy rápida (optimizado para el hardware)
- Código fuente protegido (no se distribuye)

Desventajas:

- Plataforma-dependiente (compilar para Windows ≠ para Linux)
- Cada modificación requiere recompilar

Lenguajes Interpretados

No generan código objeto. El código fuente se interpreta directamente, línea a línea, y se ejecuta simultáneamente. Esto se realiza mediante un programa auxiliar llamado **intérprete**, que debe estar cargado en memoria. La ejecución es menos eficiente que los compilados. La ventaja es que el código fuente se interpreta directamente sin un paso de compilación explícito a código objeto. La detección de errores es más detallada. La principal desventaja es que el código fuente es legible, lo que puede comprometer la seguridad o los derechos de autor. Ejemplos incluyen PHP y JavaScript.

Proceso:

Código Fuente (.js) → Intérprete → Ejecución directa línea a línea

Ventajas:

- Multiplataforma (el intérprete existe en cada SO)
- Desarrollo rápido (no esperar a compilar)
- Depuración fácil (error en la línea exacta)

Desventajas:

- Ejecución más lenta
- Código fuente visible (problemas de seguridad)

Lenguajes Mixtos o de Máquina Virtual

Combinan características de ambos. El código fuente se compila a un código binario intermedio denominado **bytecode**. Este bytecode puede considerarse código objeto, pero está destinado a una **máquina virtual** en lugar de a código objeto nativo. Luego, este bytecode se interpreta para ejecutarlo en cualquier máquina virtual compatible. Son más portables que los lenguajes compilados. El objetivo es compilar una vez y ejecutar en distintos sistemas. Ejemplos incluyen Java, C# o Python.

Proceso:

Código Fuente (.java) → Compilador → Bytecode (.class) → JVM → Ejecución

💡 **Ejemplo real:** Cuando descargas una app de Android, no descargas código Java, sino bytecode (.dex) que la Dalvik/ART VM ejecuta. Esto permite que la misma app funcione en cualquier dispositivo Android.

La ventaja de compilar a código intermedio (IL en C#, bytecode en Java) es la portabilidad: el mismo archivo .dll puede ejecutarse en Windows, Linux o Mac siempre que haya una máquina virtual instalada. No necesitas recompilar para cada sistema operativo.



5.2.3. Según su sistema de tipos (Tipado Fuerte, Tipado Débil)

Un **tipo de dato** es una clasificación que define el conjunto de valores que una variable puede tomar y las operaciones válidas que se pueden realizar sobre esos valores. Esta clasificación es fundamental porque **determina la cantidad de memoria que el sistema operativo debe reservar** para la variable. Por ejemplo, una variable de tipo int ocupará menos espacio que una de tipo float porque su rango de valores es menor.

El **sistema de tipos** de un lenguaje de programación es un conjunto de reglas que definen cómo se manejan y verifican estos tipos de datos. La clasificación de estos sistemas se basa en tres dimensiones principales: la rigidez, el momento de verificación y la declaración.

. Rigidez: Tipado Fuerte vs. Tipado Débil

Esta dimensión se refiere a la flexibilidad con la que un lenguaje maneja las conversiones entre tipos de datos.

- **Lenguajes de Tipado Fuerte**: Requieren que los tipos de datos sean compatibles para realizar operaciones. No permiten conversiones automáticas o "implícitas" entre tipos no relacionados. Si intentas sumar un número y una cadena de texto, el lenguaje lo marcará como un error, lo que previene errores inesperados en tiempo de ejecución y hace el código más robusto.
    - **Ejemplos**: C#, Java, Python, Ruby.

```
// C# (tipado fuerte) - No permite asignar un string a un int
int resultado = "hola";  // Error de compilación: Cannot implicitly convert type 'string' to 'int'
```

```
# Python (tipado fuerte)
resultado = 5 + "3"  # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

- **Lenguajes de Tipado Débil**: Permiten conversiones de tipo automáticas. El lenguaje puede intentar convertir un tipo de dato a otro sin que el programador lo solicite explícitamente. Esta flexibilidad puede llevar a errores que son difíciles de detectar.
    - **Ejemplos**: JavaScript, PHP, VBScript.

```
// JavaScript (tipado débil)
resultado = "5" + 3  // "53" (convierte 3 a string)
resultado = "5" - 3  // 2 (convierte "5" a número)
```

5.2.4. Según la forma en que operan (Paradigmas de Programación)

Un **paradigma de programación** es un modelo fundamental para el diseño y la implementación de programas, que determina la estructura y el enfoque del código. La mayoría de los lenguajes de programación modernos, como **Java**, **Kotlin** y **C#**, son multiparadigma, lo que significa que combinan características de varios de ellos para ofrecer mayor flexibilidad y poder a los desarrolladores.

Principales Paradigmas

- **Programación Estructurada**: Se basa en una serie de comandos que la computadora ejecuta en orden para cambiar el estado del programa. Es un enfoque muy directo y se basa en tres estructuras principales: sentencias secuenciales, selectivas (condicionales) y repetitivas (bucles). Es fácil de entender para programas sencillos, pero puede volverse difícil de manejar en proyectos grandes, ya que todo el código se concentra en un solo bloque. Ejemplos: C, Pascal.

```
// Programación estructurada en C
int main() {
    int i, suma = 0;
    for (i = 1; i <= 10; i++) {  // Bucle (repetitiva)
        if (i % 2 == 0) {         // Condicional (selectiva)
            suma += i;             // Secuencial
        }
    }
    return 0;
}
```

}

- **Programación Procedimental**: Es un subtipo del paradigma estructurado. Aquí, los programas se organizan en **procedimientos** (o funciones) que manipulan el estado global del programa. Se relaciona estrechamente con la programación estructurada y modular. C, Visual Basic, y otros lenguajes permiten este estilo de programación al escribir funciones o métodos que no están necesariamente asociados a una clase.

- **Programación Orientada a Objetos (POO)**: En este paradigma, los programas se construyen como una colección de **objetos** que interactúan entre sí. Un objeto es una instancia de una **clase** que contiene datos (atributos) y los métodos para operar sobre ellos. La POO promueve la reutilización de código, facilita la depuración y mejora el mantenimiento a largo plazo. Sus pilares son el **polimorfismo**, la **herencia** y la **encapsulación**. Ejemplos: C#, C++, Java, Kotlin.

```
// POO en C#
class Coche
{
    public string Marca { get; set; }     // Propiedad
    public string Modelo { get; set; }    // Propiedad

    public Coche(string marca, string modelo)  // Constructor
    {
        Marca = marca;
        Modelo = modelo;
    }

    public void Arrancar()               // Método
    {
        Console.WriteLine($"{Marca} {Modelo} arrancado");
    }
}

var miCoche = new Coche("Toyota", "Corolla");  // Objeto
miCoche.Arrancar();                             // Llamada a método
```

- **Programación Declarativa**: Los programas describen el **resultado deseado**, no el proceso paso a paso para lograrlo. Suelen ser lenguajes interpretados.
    - **Funcional**: Se enfoca en el uso de **funciones matemáticas** que no cambian el estado ni los datos externos. Esto resulta en un código modular y estructurado, aunque puede volverse complejo. C# (con LINQ y expresiones lambda) y Java (con Streams) han incorporado características importantes de este paradigma. Kotlin está diseñado con el paradigma funcional en mente desde el principio. Ejemplos: SQL, Lisp, Prolog,..

SQL - Select

```
SELECT nombre, edad FROM empleados
ORDER BY edad DESC;
```

- **Programación Multiparadigma**: Son lenguajes que admiten y combinan múltiples paradigmas. Esto permite a los desarrolladores elegir el mejor enfoque para cada parte del problema. Ejemplos: C++, JavaScript, Python, Java, Kotlin, C#.

Paradigma	Idea central	Lenguajes	Ejemplo de uso
Imperativo	Secuencia de pasos	C, Pascal	Scripts, sistemas

Paradigma	Idea central	Lenguajes	Ejemplo de uso
POO	Objetos con datos y métodos	Java, C#, C++	Apps empresariales
Declarativo	Describes qué, no cómo	SQL, HTML	Bases de datos, web
Multiparadigma	Combina varios	C#, Python, JavaScript	La mayoría de proyectos modernos

5.3. Criterios para la Selección de un Lenguaje de Programación

La elección del lenguaje a utilizar en un proyecto es de extrema importancia. Algunos criterios para su selección son:

Criterio	Pregunta clave
Campo de aplicación	¿Para qué tipo de aplicación? (web, móvil, escritorio, ciencia)
Experiencia del equipo	¿Ya conocen el lenguaje?
Herramientas disponibles	¿Hay buenos IDEs, frameworks, librerías?
Documentación	¿Hay buena documentación oficial?
Comunidad	¿Hay gente que pueda ayudar en foros/StackOverflow?
Rendimiento	¿Necesita alta velocidad?
Reusabilidad	¿Se puede usar código existente?
Portabilidad	¿Debe funcionar en varias plataformas?
Coste	¿Licencias, herramientas, formación?
Imposición del cliente	¿El cliente exige algo específico?

📝 **Nota:** No existe el "mejor" lenguaje. Existe el lenguaje adecuado para cada situación. Un científico de datos prefiere Python por sus librerías de ML. Un desarrollador de videojuegos AAA prefiere C++ por rendimiento. Un startup web elige JavaScript/Node.js por velocidad de desarrollo.

Lenguajes como Java, C, C++, PHP y Visual Basic concentran alrededor del 60% del interés de la comunidad informática mundial. Existen índices como TIOBE que analizan la demanda de lenguajes de programación diariamente.

Lenguajes por área profesional:

Área	Lenguajes típicos
Desarrollo Web Frontend	JavaScript, TypeScript
Desarrollo Web Backend	PHP, Python, Java, Node.js, Ruby, C#

Área	Lenguajes típicos
Desarrollo Móvil	Swift (iOS), Kotlin/Java (Android), Dart (Flutter)
Ciencia de Datos	Python, R, Julia
Sistemas Embebidos	C, C++, Rust
Videojuegos	C++, C#, Lua
DevOps/Scripting	Python, Bash, Go
Inteligencia Artificial	Python, Lisp, Prolog

C# se usa en: desarrollo web con ASP.NET, escritorio con WPF y MAUI, juegos con Unity, móviles con .NET MAUI, y servicios en la nube con Azure. Es uno de los lenguajes más versátiles del mercado.

Resumen del punto:

Concepto	Descripción
Bajo nivel	Máquina, ensamblador (difícil, rápido, no portable)
Medio nivel	C (acceso a memoria, más abstracto)
Alto nivel	Python, Java, JavaScript (fácil, portable)
Compilado	C, C++ (rápido, platforma-dependiente)
Interpretado	Python, JavaScript (lento, multiplataforma)
Mixto	Java, C# (bytecode + máquina virtual)
Tipado fuerte	Python, Java (seguro, detecta errores)
Tipado débil	JavaScript, PHP (flexible, propenso a bugs)
POO	Java, C#, Python (reutilización, mantenimiento)
Funcional	Haskell, JavaScript moderno (funciones puras)

En el siguiente punto veremos el **proceso de traducción**, es decir, cómo los lenguajes de alto nivel se convierten en código que la máquina puede ejecutar.

6. PROCESO DE TRADUCCIÓN, MÁQUINAS VIRTUALES Y ENTORNOS DE EJECUCIÓN

6.1. Proceso de Traducción: Compilación e Interpretación

Para que el ordenador entienda algo escrito en un lenguaje de programación, debe pasar por un proceso de traducción de código. La traducción de un programa escrito en un lenguaje de programación a un lenguaje de máquina se realiza mediante un **traductor**, que puede ser un **compilador** o un **intérprete**.

💡 **Analogía:** Imagina que tienes un libro en japonés y quieres leerlo. Tienes dos opciones:

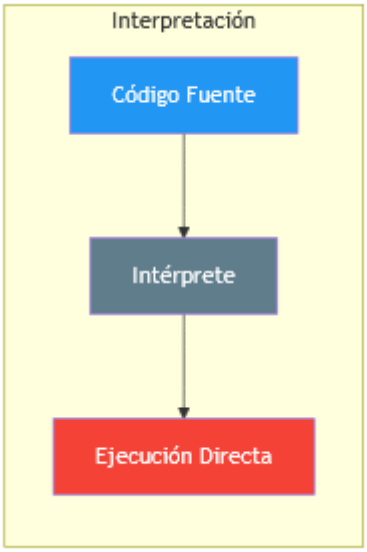
- **Compilar:** Traducir TODO el libro al español antes de leerlo (trabajo largo antes, lectura rápida después)
- **Interpretar:** Leerlo con un traductor que te va traduciendo frase por frase mientras lees (más lento pero adaptable)

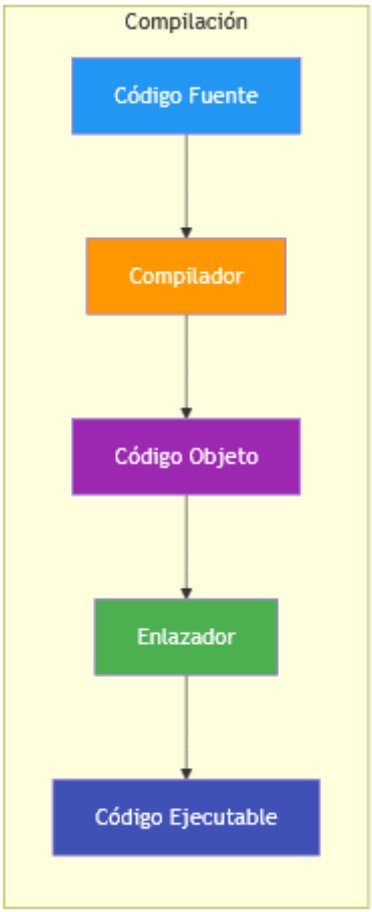
6.1.1. Diferenciación entre Traducción, Compilación e Interpretación



- **Traducción**: Es el proceso general de transformar código de un lenguaje a otro.
- **Compilación**: Proceso que traduce el código fuente completo a código objeto o binario ejecutable en un solo paso. Un ejemplo es el compilador de C.
- **Interpretación**: Proceso que traduce y ejecuta el código fuente línea a línea, o instrucción por instrucción, sin generar un archivo intermedio. Un ejemplo es el intérprete de JavaScript.
- **Mixto**: Algunos lenguajes utilizan ambos métodos, compilando a un código intermedio (bytecode) que luego es interpretado por una máquina virtual. Un ejemplo es Java y C#.


LENGUAJES DE PROGRAMACIÓN
SEGÚN SU COMPILACIÓN
La compilación es el proceso de traducir el código de un lenguaje de alto nivel al lenguaje máquina que leen las computadoras.
COMPILADOS
Se convierten en código binario a través de un compilador.
1 Escribes el código.
2 Pasa por un proceso de compilación que genera un archivo ejecutable.
3 Ese ejecutable es leído por la computadora.
EXE
INTERPRETADOS
Requieren de un programa que lee la instrucción en tiempo real y la ejecuta.
1 Escribes el código.
2 En la computadora, el programa interpreta el código y lo traduce a lenguaje máquina en tiempo real.
Domina la tecnología y las carreras del futuro en:
ed.team/cursos
EDteam


Interpretación
Código Fuente
Intérprete
Ejecución Directa


Compilación
Código Fuente
Compilador
Código Objeto
Enlazador
Código Ejecutable

💡 **Ejemplo real:** En Java: MiApp.java → javac (compilador) → MiApp.class (bytecode) → java (JVM) → Ejecución en Windows, Linux, Mac.

📝 **Nota:** Python funciona así. Cuando ejecutas python programa.py, Python compila a bytecode (.pyc) y luego lo interpreta. Por eso la segunda ejecución es más rápida.

JSX porque es más legible, y Babel lo convierte en llamadas a React.createElement() que el navegador entiende.

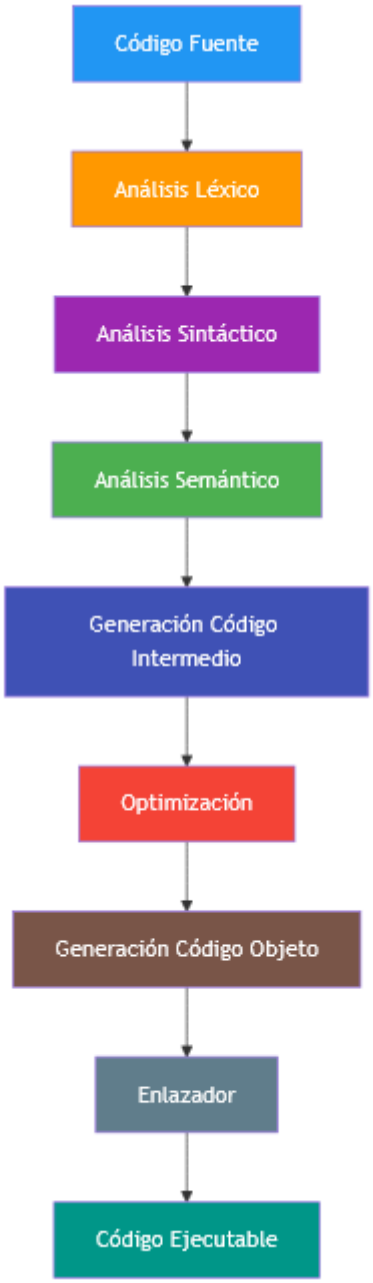
6.1.2. Fases de un Traductor (Compilador/Intérprete)

Un **traductor** es un programa que convierte el código escrito por un programador (código fuente) en un lenguaje que la máquina puede entender directamente (código máquina o código objeto). Este proceso no es una simple traducción palabra por palabra, sino que se lleva a cabo en varias fases bien definidas.

.

Antes de producir el código máquina final, muchos compiladores generan un **código intermedio**. Este es un lenguaje de bajo nivel, parecido al ensamblador, pero independiente de la arquitectura de la máquina de destino. Esta fase simplifica el diseño del compilador, ya que las optimizaciones pueden realizarse sobre este código genérico en lugar

.



Generación de Código Objeto

En esta fase, el código intermedio (ya optimizado) se convierte en **código máquina** de la arquitectura específica (por ejemplo, x86, ARM). El resultado es un archivo binario que contiene instrucciones que la CPU puede ejecutar directamente. Sin embargo, este código aún no es un programa completo, ya que las referencias a funciones o datos de otras partes del programa o de librerías externas están representadas por etiquetas simbólicas.

Enlazador (Linker) y Cargador (Loader)

- **Enlazador (Linker):** Es el programa que toma uno o más archivos de código objeto y los combina con las **librerías** y rutinas necesarias (como las funciones para entrada y salida) para crear un único **archivo ejecutable** completo. El enlazador resuelve las referencias simbólicas, asignando direcciones de memoria reales. En lenguajes como C, esto incluye las instrucciones del preprocesador (ej. #include), que se encargan de incluir el contenido de otros archivos antes de la compilación..
- **Cargador (Loader):** Aunque no es parte del compilador, es la fase final que se encarga de cargar el archivo ejecutable en la memoria RAM y prepara su ejecución cuando el usuario lo inicia.

6.2. Códigos Fuente, Objeto y Ejecutable

Durante el proceso de codificación, el código pasa por diferentes estados:

- **Código Fuente**: Es el archivo de texto legible escrito por los programadores en un lenguaje de programación de alto nivel. Contiene el conjunto de instrucciones necesarias. Este código no es directamente ejecutable por la máquina y debe ser traducido. Un aspecto importante es su licencia: puede ser **abierto** (disponible para estudiar, modificar, reutilizar) o **cerrado** (no se tiene permiso para editarlo).
- **Código Objeto (Intermedio)**: Es un archivo binario no ejecutable. Es el resultado de traducir (compilar) el código fuente a un código equivalente formado por unos y ceros. En Java, el código objeto se denomina **Bytecode**. Solo existe si el programa se compila. No es directamente inteligible por el ser humano ni por la computadora.
- **Código Ejecutable**: Es el archivo binario ejecutable directamente por la computadora. También conocido como **código máquina**. Se obtiene al enlazar los archivos de código objeto con ciertas rutinas y bibliotecas necesarias. El sistema operativo es el encargado de cargarlo en memoria RAM y ejecutarlo. Los programas interpretados no producen código objeto, el paso de fuente a ejecutable es directo.

📝 **Nota:** Extensiones típicas:

- .c, .cpp, .java, .py, .js, .php (código fuente)
- .o, .obj (código objeto)
- .exe, .app, .bin (ejecutable)

💡 **Dato:** Cuando desarrollas en Java, trabajas con código fuente (.java). El compilador javac genera bytecode (.class). Cuando ejecutas java MiClase, la JVM carga el bytecode y lo interpreta/JIT-compila a código máquina nativo.

6.3. Máquinas Virtuales y Entornos de Ejecución

6.3.1. Concepto de Máquina Virtual

Una **máquina virtual (MV)** es un tipo especial de software cuya misión es separar el funcionamiento del ordenador de los componentes hardware instalados. Actúa como una capa de software de bajo nivel, haciendo de puente entre el bytecode de la aplicación y los dispositivos físicos del sistema. Esto garantiza la **portabilidad** de las aplicaciones, permitiendo desarrollarlas y ejecutarlas sobre cualquier equipo, independientemente de sus características hardware.

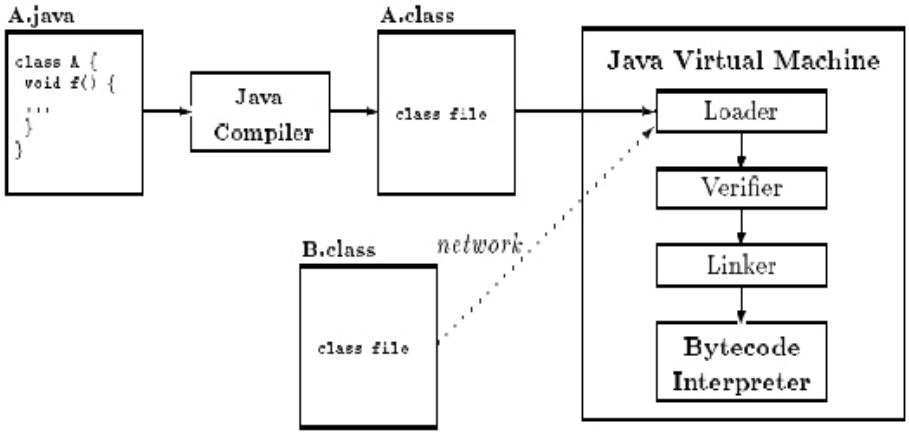
💡 **Analogía:** La máquina virtual es como un traductor universal que permite que tu programa hable "java" con el hardware que solo entiende "máquina".

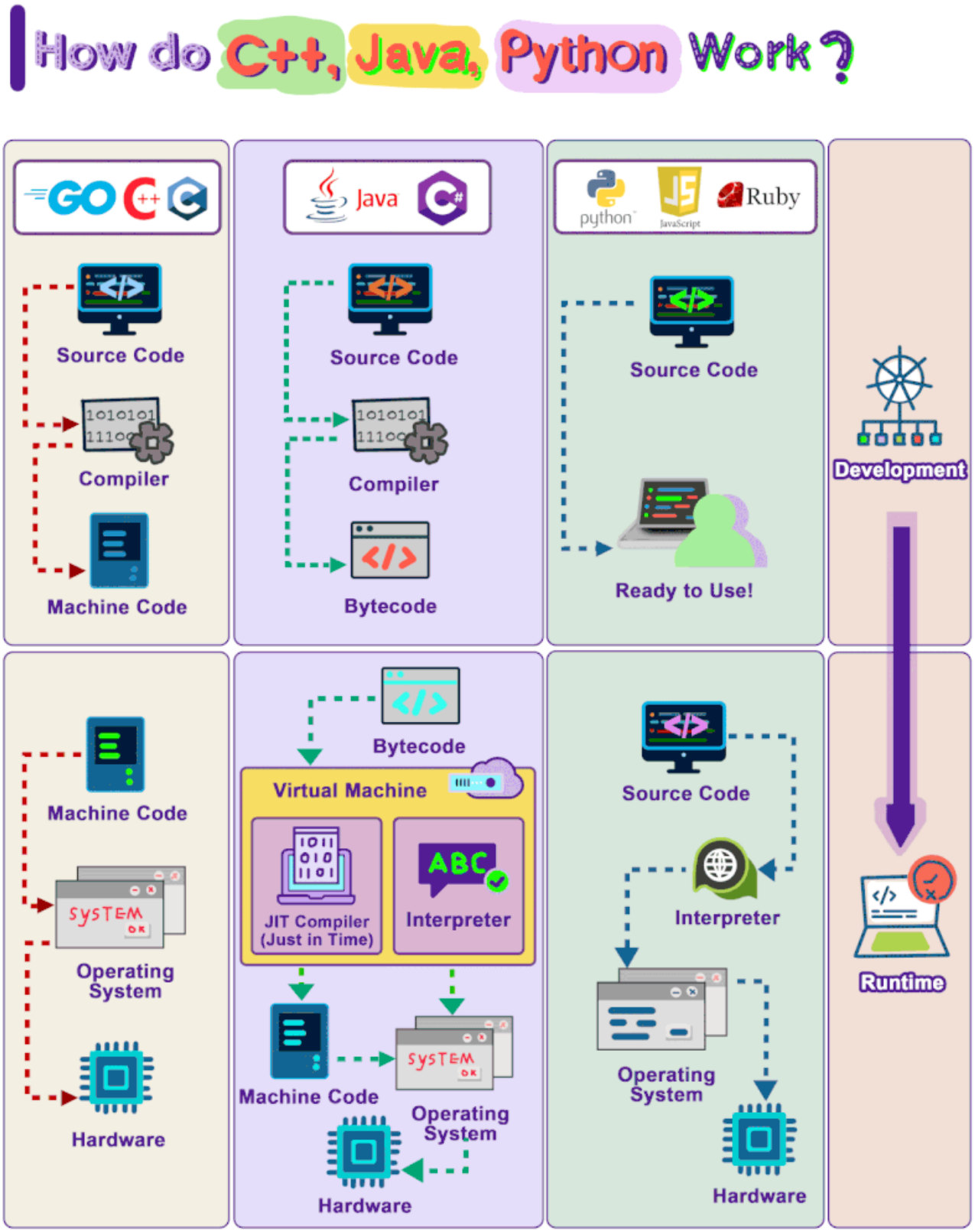
Funciones principales de una máquina virtual

- **Portabilidad:** Ejecutar el mismo código en diferentes sistemas operativos
- **Gestión de memoria:** Reservar y liberar memoria automáticamente (garbage collector)
- **Seguridad:** Aislar aplicaciones entre sí
- **Verificación:** Comprobar bytecode antes de ejecutar

Los **lenguajes mixtos o virtuales** (como Java o Python) compilan el código fuente a un código intermedio llamado **bytecode** (en Java), que luego es interpretado por la máquina virtual.

Sin máquinas virtuales, tendrías que compilar tu aplicación por separado para Windows, Linux y Mac. Con una MV, compilas una sola vez a código intermedio y el mismo ejecutable funciona en cualquier sistema operativo que tenga la MV instalada. Es como el traductor universal: tú hablas una vez y él traduce a cualquier idioma.





6.3.2. Entornos de Ejecución (Runtime Environments)

Un **entorno de ejecución** es un servicio de máquina virtual que sirve como base software para la ejecución de programas. Puede pertenecer al sistema operativo o instalarse como software independiente. Es un conjunto de utilidades que permiten la ejecución de programas. Se encarga de configurar la memoria principal, enlazar los

archivos del programa con bibliotecas existentes y subprogramas creados, y depurar programas (comprobar errores semánticos).

El Entorno de Ejecución está formado por la máquina virtual y los **API's** (bibliotecas de clases estándar), que se distribuyen conjuntamente al necesitar ser compatibles. Funciona como intermediario entre el lenguaje fuente y el sistema operativo, ejecutando aplicaciones.

Entornos de ejecución comunes:

Entorno	Lenguaje	Propósito
JRE (Java Runtime Environment)	Java	Ejecutar aplicaciones Java
.NET 8/9/10 Runtime	C#	Ejecutar aplicaciones .NET modernas
Node.js	JavaScript	Ejecutar JS en servidor
Python Runtime	Python	Ejecutar scripts Python

📝 **Nota:** Cuando instaláis Python, estáis instalando el intérprete + la biblioteca estándar + el runtime. Sin esto, no podríais ejecutar programas .py.

6.3.3. Frameworks

Un **framework** (plataforma, entorno, marco de trabajo de desarrollo rápido de aplicaciones) es una estructura de ayuda para el programador, en base a la cual se pueden desarrollar proyectos sin partir desde cero. Es una plataforma software que define programas de soporte, bibliotecas, lenguajes interpretados, etc., ayudando a desarrollar y unir los diferentes módulos de un proyecto.

💡 **Analogía:** Un framework es como un kit de construcción de muebles IKEA. No tienes que diseñar las piezas desde cero, sigues las instrucciones del kit y lo montas.

Ventajas de utilizar un framework

- **Desarrollo rápido de software**.
- **Reutilización de partes de código** para otras aplicaciones.
- **Diseño uniforme** del software.
- **Portabilidad de aplicaciones**, ya que los bytecodes generados pueden ser ejecutados sobre cualquier máquina virtual compatible.

Inconvenientes

- **Curva de aprendizaje:** Tiempo para aprender el framework
- **Dependencia:** El código está acoplado al framework
- **Rendimiento:** Puede añadir overhead

Ejemplos de Frameworks son **.NET** (para Windows, con el ".Net framework" para la ejecución) y **Spring de Java** (conjuntos de bibliotecas para desarrollo y ejecución de aplicaciones).

Frameworks por lenguaje:

Lenguaje	Frameworks populares
Java	Spring, Jakarta EE, Vaadin
Python	Django, Flask, FastAPI
JavaScript	React, Vue, Angular, Node.js
PHP	Laravel, Symfony, WordPress
C#	.NET, ASP.NET, Entity Framework

Resumen del punto:

Concepto	Descripción
Compilación	Traduce TODO el código → ejecutable (rápido, C/C++)
Interpretación	Traduce y ejecuta línea a línea (lento, Python/JS)
Mixto	Compila a bytecode → máquina virtual (Java, C#)
Análisis léxico	Código → tokens
Análisis sintáctico	Tokens → árbol (¿es correcto?)
Análisis semántico	Árbol → ¿tiene sentido?
Código fuente	Tu archivo .java, .py
Código objeto	.class, .o (no ejecutable)
Código ejecutable	.exe, .app (la máquina lo entiende)
Máquina virtual	Capa que hace portable el código (JVM, CLR)
Framework	Kit de construcción reutilizable

En el siguiente punto veremos las **herramientas de apoyo** al desarrollo: editores, IDEs y otras utilidades que facilitan nuestro trabajo diario como programadores.

7. HERRAMIENTAS DE APOYO AL DESARROLLO DE SOFTWARE

7.1. Herramientas de Desarrollo

En la práctica, para llevar a cabo varias de las etapas del desarrollo de software, se utilizan **herramientas informáticas**. Su finalidad principal es automatizar las tareas y ganar fiabilidad y tiempo. Esto permite a los desarrolladores centrarse en los requerimientos del sistema y el análisis, que son las causas principales de los fallos del software. Los tipos de software de desarrollo incluyen editores, compiladores e intérpretes.

💡 **Analogía:** Las herramientas de desarrollo son como los instrumentos de un mecánico. Puedes cambiar una rueda con una llave inglesa básica, pero con las herramientas adecuadas el trabajo es más rápido, seguro y profesional.

Categorías de herramientas:

Categoría	Herramientas populares	Función
Editores	Visual Studio Code, Netbeans, Eclipste	Escribir código
Compiladores	dotnet (C#), javac (Java), gcc (C/C++)	Traducir código
Intérpretes	dotnet run, node, Python, JVM	Ejecutar directamente
Depuradores	Visual Studio Debugger, Chrome DevTools	Encontrar errores
Control de versiones	Git, SVN, Mercurial	Gestionar cambios
Gestores de paquetes	NuGet, npm, maven, gradle	Instalar librerías
Testing	JUnit, pytest, Jest	Verificar código

7.2. Herramientas CASE (Computer Aided Software Engineering)

Las **herramientas CASE** son un conjunto de aplicaciones que se utilizan en el desarrollo de software con el objetivo de reducir costes y tiempo del proceso, mejorando la productividad.

💡 **Nota:** CASE significa "Ingeniería de Software Asistida por Computadora", como CAD (Diseño Asistido por Computadora) pero para software.

Funcionalidad:

- **Automatización de tareas repetitivas** en análisis, diseño y codificación
- **Generación automática de código** a partir de modelos
- **Creación de diagramas** (UML, ER, flujo)
- **Validación de requisitos** y consistencia
- **Documentación automática** del proyecto
- **Gestión de la configuración** del software

Herramientas CASE gratuitas/libres:

- **ArgoUML:** http://argouml.tigris.org/ - Herramienta UML open source
- **Dia:** http://dia-installer.de/ - Diagramas diversos
- **StarUML:** https://staruml.io/ - UML con versión gratuita
- **PlantUML:** https://plantuml.com/ - UML mediante texto

7.4. Entornos de Desarrollo Integrado (IDE)

Un **Entorno de Desarrollo Integrado (IDE)** es una herramienta que facilita y posibilita el desarrollo de software. Agrupa diversas herramientas de desarrollo (editor de código, compilador, depurador) en una única interfaz gráfica para aumentar la productividad del programador.

Componentes de un IDE:

Componente	Descripción
Editor avanzado	Resaltado, sangrado, navegación
Compilador integrado	Traducir sin salir del IDE
Depurador visual	Puntos de ruptura, inspección
Autocompletado	Sugerencias de código
Refactorización	Renombrar, extraer métodos
Gestor de proyectos	Organizar archivos
Control de versiones	Git integrado

IDEs populares por lenguaje:

Lenguaje	IDE principal	Alternativas
C#	JetBrains Rider	Visual Studio Code, Visual Studio
Java	IntelliJ IDEA	Eclipse, NetBeans
Python	PyCharm	VS Code, Spyder
C/C++	CLion	Visual Studio, VS Code
JavaScript	WebStorm	VS Code, Atom
General	Visual Studio Code	Sublime

Extensiones esenciales para VS Code (DAW):

Extensión	Utilidad
C# Dev Kit	Soporte completo para C#/.NET
Prettier	Formateo de código
ESLint	Linting JavaScript
Python (Microsoft)	Soporte Python
Java (Red Hat)	Soporte Java
Live Server	Servidor local para web
GitLens	Mejora Git
Material Icon Theme	Iconos atractivos

💡 **Dato:** El primer IDE fue "Eclipse" (1999), desarrollado por IBM para Java. Antes, los programadores editaban archivos de texto en terminales y compilaban manualmente.

7.5. Control de Versiones: Git

Git es la herramienta de control de versiones más utilizada en el mundo. Permite registrar los cambios realizados en archivos a lo largo del tiempo, de modo que puedas recuperar versiones anteriores y trabajar en equipo sin pisarte los unos a los otros.

💡 **Analogía:** Git es como el "Historial" de Google Docs, pero mucho más potente. Cada "guardado" (commit) es una fotografía de tu proyecto en ese momento. Si algo se rompe, puedes volver a la foto anterior.

📝 **Nota:** En DAW vais a usar Git en todos los proyectos. Es una habilidad fundamental en cualquier empresa de software. GitHub y GitLab son las plataformas más populares para alojar repositorios remotos.

7.6. Contenedores: Docker

Docker es una plataforma que permite empaquetar una aplicación junto con todas sus dependencias (librerías, configuraciones, bases de datos) en un "contenedor" que se ejecuta de forma idéntica en cualquier ordenador.

💡 **Analogía:** Docker es como una "caja de embalar" para software. Metes tu aplicación con todo lo que necesita dentro de la caja, y esa caja funciona igual en tu portátil, en el servidor de la empresa o en la nube. El problema clásico "en mi ordenador funciona" desaparece.

Conceptos clave:

Concepto	Descripción
Contenedor	Instancia aislada de una aplicación con todo su entorno

Concepto	Descripción
Imagen	Plantilla de solo lectura que define el contenedor (como un "molde")
Dockerfile	Archivo de texto con instrucciones para crear la imagen
Docker Hub	Repositorio público de imágenes (como GitHub pero para contenedores)

¿Por qué Docker en desarrollo?

- **Reproducibilidad**: Todo el equipo trabaja con el mismo entorno
- **Aislamiento**: Cada servicio (app, base de datos, caché) corre en su propio contenedor
- **Portabilidad**: Funciona igual en Windows, Linux y Mac
- **Rapidez**: Los contenedores arrancan en segundos, no en minutos como una máquina virtual

📝 **Nota:** Docker no reemplaza a las máquinas virtuales, pero para desarrollo y despliegue de aplicaciones web es mucho más ligero y rápido. En proyectos .NET, es habitual usar Docker para levantar la base de datos (SQL Server, PostgreSQL) sin instalarla en tu ordenador.

8. PERFILES DEL DESARROLLO DE SOFTWARE

El desarrollo de software es un proceso que involucra a diferentes profesionales, cada uno con roles y responsabilidades específicas a lo largo del ciclo de vida del software. Estos roles son cruciales para el éxito de un proyecto, combinando conocimientos técnicos, de gestión y de negocio.

💡 **Analogía:** Un equipo de desarrollo de software es como una orquesta. Hay diferentes instrumentos (roles) que deben tocar juntos bajo la dirección de un director (jefe de proyecto) para crear una sinfonía coherente.

8.1. Arquitecto de Software

- Este profesional tiene la responsabilidad de decidir "cómo" se realiza el proyecto y cómo se estructurará.
- Posee un amplio conocimiento de las tecnologías, los *frameworks* y las librerías disponibles.
- Decide y conforma los recursos necesarios para el desarrollo de un proyecto.
- Se involucra activamente en la fase de diseño.

Responsabilidades principales:

- Diseñar la arquitectura general del sistema
- Elegir tecnologías, frameworks y patrones
- Definir estándares y convenciones técnicas
- Tomar decisiones de alto impacto técnico
- Revisar código y diseños de otros desarrolladores

Skills necesarios:

- Amplia experiencia (5-10+ años)
- Conocimiento profundo de múltiples tecnologías
- Capacidad de diseño y abstracción
- Comunicación con stakeholders técnicos y no técnicos

📝 **Nota:** En DAW seréis programadores, pero con experiencia podréis crecer hacia roles de arquitectura. El arquitecto es el "veterano" del equipo técnico.

8.2. Jefe de Proyecto

- Es el encargado de dirigir el curso del proyecto.
- Puede ser un analista con experiencia, un arquitecto o una persona dedicada en exclusividad a este puesto.
- Debe poseer habilidades para gestionar un equipo y lidiar con los tiempos y plazos.
- Mantiene una comunicación continua y fluida con el cliente.

Responsabilidades principales:

- Planificar el proyecto (tiempo, recursos, presupuesto)
- Coordinar al equipo de desarrollo

- Gestionar expectativas del cliente
- Identificar y mitigar riesgos
- Reporting a dirección y stakeholders

Skills necesarios:

- Gestión de proyectos (PM, Agile, Scrum)
- Comunicación interpersonal
- Resolución de conflictos
- Gestión del tiempo
- Conocimientos técnicos (para entender al equipo)

💡 **Dato:** Muchos jefes de proyecto en software provienen de perfiles técnicos (ex-programadores) porque entienden mejor las complejidades del desarrollo.

8.3. Analista de Sistemas

- Realiza un estudio exhaustivo del problema a resolver.
- Efectúa el análisis y el diseño de todo el sistema.
- Este perfil requiere mucha experiencia y suele involucrarse en reuniones con el cliente para recopilar requisitos.
- Es la figura clave en la fase de **Análisis**, donde se determinan y definen las necesidades del cliente y los requisitos del software. También interviene en la fase de diseño.

Responsabilidades principales:

- Reuniones con el cliente para entender necesidades
- Documentar requisitos funcionales y no funcionales
- Crear casos de uso y historias de usuario
- Diseñar la estructura del sistema
- Crear documentación técnica

Skills necesarios:

- Pensamiento analítico
- Comunicación con clientes no técnicos
- Conocimiento de metodologías (UML, BPMN)
- Experiencia en el dominio del negocio

📝 **Nota:** El analista es el "traductor" entre lo que quiere el cliente (lenguaje de negocio) y lo que necesita el programador (lenguaje técnico). Es crucial para evitar malentendidos.

8.4. Analista Programador

- Según las fuentes, este rol comparte muchas responsabilidades con el **Analista de Sistemas**, incluyendo la realización de un estudio exhaustivo del problema, la ejecución del análisis y diseño del sistema, y la interacción con el cliente.
- Este perfil también requiere mucha experiencia y conocimiento tanto en la definición de soluciones como en la capacidad de comprender la implementación técnica.

Responsabilidades principales:

- Combinar análisis técnico con programación
- Diseñar módulos y componentes específicos
- Implementar funcionalidades complejas
- Revisar código de otros programadores
- Mentorizar a programadores junior

Skills necesarios:

- Fuertes habilidades técnicas (programación)
- Capacidad de análisis y diseño
- Comunicación con analistas y programadores
- Conocimiento del negocio

💡 **Consejo:** El analista se centra en el "qué" (requisitos), el programador en el "cómo" (implementación). El analista-programador hace ambas cosas.

8.5. Programador (o Desarrollador)

- Conoce en profundidad el lenguaje de programación que se utiliza en el proyecto.
- Se encarga de codificar las tareas encomendadas por el analista o el analista programador.
- Su misión principal es la de codificar y probar los diferentes módulos de la aplicación.
- Actúa en la fase de **Codificación (Implementación)**, escribiendo el código fuente y traduciendo los algoritmos a un lenguaje de programación.
- No suele intervenir en la fase de diseño.

Niveles en la carrera del programador:

Nivel	Años experiencia	Responsabilidad
Junior	0-2	Tareas simples, supervisión necesaria
Mid-level	2-5	Tareas independientes, mentoring juniors
Senior	5-10+	Diseño técnico, decisiones complejas
Lead/Tech Lead	8+	Liderazgo técnico del equipo

Skills necesarios:

- Dominio de al menos un lenguaje de programación
- Conocimiento de frameworks y librerías
- Resolución de problemas
- Lectura y escritura de código limpio
- Control de versiones (Git)
- Testing básico

Carrera profesional típica desde DAW:

💡 **Consejo:** En DAW empezaréis como Junior. No os preocupéis por el nivel: con experiencia y práctica, el crecimiento es natural. Lo importante es nunca dejar de aprender.

8.6. QA (Quality Assurance) / Testeador

- Aunque las fuentes no lo mencionan explícitamente como un "rol" con título específico en la lista de perfiles, la fase de **Pruebas** es fundamental y su objetivo principal es "conseguir que el programa funcione incorrectamente para descubrir y corregir defectos".
- Este rol se enfoca en someter el programa al máximo número de situaciones diferentes, realizando pruebas unitarias, de integración, funcionales, estructurales y *Beta Test*.
- La existencia de estas pruebas detalladas implica que hay personal dedicado o especializado en la validación y verificación del software construido, asegurando su calidad antes de la entrega al cliente.
- Los programadores también tienen la misión de probar los módulos que desarrollan, pero en proyectos más grandes o complejos, se suelen establecer roles dedicados a la calidad y las pruebas.

Responsabilidades principales:

- Diseñar casos de prueba
- Ejecutar pruebas manuales y automatizadas
- Reportar bugs y verificar correcciones
- Automatizar pruebas de regresión
- Definir métricas de calidad

Skills necesarios:

- Pensamiento creativo (encontrar casos límite)
- Conocimiento de herramientas de testing
- Automatización (Selenium, Cypress, etc.)
- Comunicación con desarrolladores
- Conocimiento técnico del sistema

💡 **Dato:** El testing es una carrera en sí misma. Hay QA manual, automatización de pruebas, testing de rendimiento, security testing, etc.

8.8. Scrum Master

El **Scrum Master** es el responsable de facilitar el proceso Scrum y eliminar obstáculos que impidan al equipo avanzar.

Responsabilidades:

- Facilitar los eventos de Scrum (daily, planning, review, retrospective)
- Proteger al equipo de distracciones externas
- Ayudar al Product Owner a gestionar el backlog
- Promover la mejora continua
- Eliminar impedimentos técnicos o organizativos

NO es: un jefe de proyecto. No asigna tareas ni toma decisiones técnicas. Es un facilitador y coach del equipo.

8.10. Diseñador UX/UI

Aunque no es un perfil estrictamente de programación, el diseñador UX/UI es fundamental en proyectos web y móviles:

UX (User Experience): Se centra en la experiencia del usuario. Cómo se siente el usuario al usar la aplicación, si encuentra lo que busca, si el flujo es lógico.

UI (User Interface): Se centra en la apariencia visual. Colores, tipografías, botones, espaciados, iconografía.

Responsabilidades principales:

- Investigar necesidades del usuario (entrevistas, encuestas)
- Crear wireframes (bocetos de baja fidelidad)
- Diseñar prototipos interactivos (Figma, Adobe XD)
- Definir el sistema de diseño (componentes reutilizables)
- Realizar testing de usabilidad

Skills necesarios:

- Herramientas de diseño (Figma, Sketch, Adobe XD)
- Conocimiento de principios de usabilidad
- Prototipado interactivo
- Comunicación con desarrolladores
- Conocimiento básico de HTML/CSS

💡 **Dato:** En DAW, muchos programadores terminan haciendo tareas de UI porque entienden las restricciones técnicas. Un diseñador que conoceHTML/CSS y un programador que entiende de UX forman un equipo imparable.

8.11. Organigrama de un Equipo de Desarrollo

📝 **Nota:** En empresas pequeñas o startups, una persona puede acumular varios roles (programador + QA + DevOps). En empresas grandes, cada rol está especializado. En DAW vais a aprender los fundamentos de todos estos roles.

Resumen del punto:

Perfil	Función principal	Experiencia
Arquitecto	Diseña la solución técnica	5-10+ años
Jefe de Proyecto	Dirige y planifica	5+ años
Analista	Define qué hacer (requisitos)	3-5+ años
Analista Programador	Analiza + programa	3-7+ años
Programador	Escribe el código	0-10+ años
QA	Garantiza la calidad	2-5+ años
DevOps	Infraestructura y despliegue	3-7+ años
Diseñador UX/UI	Experiencia y apariencia del usuario	2-7+ años

En el siguiente punto veremos un **caso de estudio** práctico: el lenguaje C# y la plataforma .NET.